Univerza v Ljubljani
Fakulteta za računalništvo in informatiko

Jan Zajc

# Svoboda na dotik: Ali je popolnoma odprtokoden telefon možen?

OSNUTEK DIPLOMSKE NALOGE

VISOKOŠOLSKI STROKOVNI ŠTUDIJSKI PROGRAM
PRVE STOPNJE
RAČUNALNIŠTVO IN INFORMATIKA

Mentor: viš. pred. dr. David Jelenc

Ljubljana, 2026

To delo je ponujeno pod licenco *Creative Commons Priznanje avtorstva-Deljenje pod enakimi pogoji 2.5 Slovenija* (ali novejšo različico). To pomeni, da se tako besedilo, slike, grafi in druge sestavine dela kot tudi rezultati diplomskega dela lahko prosto distribuirajo, reproducirajo, uporabljajo, priobčujejo javnosti in predelujejo, pod pogojem, da se jasno in vidno navede avtorja in naslov tega dela in da se v primeru spremembe, preoblikovanja ali uporabe tega dela v svojem delu, lahko distribuira predelava le pod licenco, ki je enaka tej. Podrobnosti licence so dostopne na spletni strani creativecommons.si ali na Inštitutu za intelektualno lastnino, Streliška 1, 1000 Ljubljana.

Izvorna koda diplomskega dela, njeni rezultati in v ta namen razvita programska oprema je ponujena pod licenco GNU General Public License, različica 3 (ali novejša). To pomeni, da se lahko prosto distribuira in/ali predeluje pod njenimi pogoji. Podrobnosti licence so dostopne na spletni strani `http://www.gnu.org/licenses/`.

*Besedilo je oblikovano z urejevalnikom besedil LaTeX.*

**Kandidat:** Jan Zajc
**Naslov:** Svoboda na dotik: Ali je popolnoma odprtokoden telefon možen?
**Vrsta naloge:** Diplomska naloga na visokošolskem strokovnem programu prve stopnje Računalništvo in informatika
**Mentor:** viš. pred. dr. David Jelenc

**Opis:**
Opis diplome

**Title:** Freedom at Your Fingertips: Is a Fully Open-Source Phone Possible?

**Description:**
Opis diplome v angleščini

# Kazalo

**Povzetek**

**Abstract**

**1 Uvod** **1**

**2 Pregled področja** **3**

2.1 Duopol Android/iOS . . . . . 3

2.2 Sistem iOS . . . . . 4

2.3 Sistem Android . . . . . 4

2.4 Akterji mobilnega ekosistema . . . . . 5

2.5 Alternativni Android ROM-i . . . . . 6

2.6 Mobilni Linux: postmarketOS, Ubuntu Touch in sorodni projekti . . . . . 7

2.7 Namenske Linux naprave . . . . . 8

2.8 Povzetek . . . . . 9

**3 Teoretična izhodišča** **11**

3.1 Odprtokodna in prosta programska oprema . . . . . 11

3.2 Arhitektura sistema: od zagona do uporabniškega prostora . . 13

3.3 Gonilniki, strojna programska oprema in binarni dodatki . . . 15

3.4 Odprtost, varnost in zasebnost . . . . . 15

3.5 Vzdrževanje in življenjski cikel naprav . . . . . 17

**4 Model "popolnoma odprtokodnega telefona" 19**
4.1 Idealni referenčni model . . . . . 20
4.2 Slojni pogled na odprtost telefona . . . . . 20
4.3 Kriteriji odprtosti po slojih . . . . . 21
4.4 Kriteriji praktične uporabnosti . . . . . 26
4.5 Način ocenjevanja . . . . . 28
4.6 Povzetek . . . . . 30

**5 Analiza trenutnega stanja odprtokodnih mobilnih rešitev 31**
5.1 Alternativni ROM-i za sistem Android . . . . . 32
5.2 Mobilne distribucije sistema GNU/Linux na obstoječi strojni opremi za sistem Android . . . . . 38
5.3 Namenski telefoni z operacijskim sistemom GNU/Linux . . . . 44
5.4 Primerjalna sinteza . . . . . 48

**6 Empirični del: študija primera na telefonu OnePlus 6T 57**
6.1 Cilj in zasnova študije . . . . . 58
6.2 Testna naprava . . . . . 58
6.3 Izbrani sistemi . . . . . 59
6.4 Metodologija in potek testiranja . . . . . 61
6.5 Empirični indikatorji odprtosti . . . . . 67
6.6 Omejitve študije . . . . . 69

**Literatura 71**

# Povzetek

**Naslov:** Svoboda na dotik: Ali je popolnoma odprtokoden telefon možen?

**Avtor:** Jan Zajc

placeholder

**Ključne besede:** odprtokodni telefon, odprtokodna programska oprema, pametni telefoni, mobilni operacijski sistemi, zasebnost.

# Abstract

**Title:** Freedom at Your Fingertips: Is a Fully Open-Source Phone Possible?

**Author:** Jan Zajc

placeholder

**Keywords:** open-source phone, open-source software, smartphones, mobile operating systems, privacy.

# Poglavje 1

# Uvod

placeholder

# Poglavje 2

# Pregled področja

Sodobni mobilni trg je močno koncentriran. V praksi ga skoraj v celoti obvladujeta Android in iOS, kar pomembno vpliva na možnosti nameščanja programske opreme, prilagajanja sistema in dolgoročnega vzdrževanja naprave. V tem poglavju najprej predstavimo ključne značilnosti obeh prevladujočih platform, nato pa pregledamo glavne odprtokodne pristope in njihove omejitve.

## 2.1 Duopol Android/iOS

Android in iOS predstavljata skoraj celoten trg mobilnih operacijskih sistemov. Po podatkih StatCounter, ki merijo globalni delež glede na uporabo, Android dosega približno 68 %, iOS pa približno 31 % [64]. Podobno sliko kažejo tudi tržne analize prodajnih deležev [65]. Ne glede na manjša odstopanja med posameznimi viri je ključna ugotovitev, da sta v praksi relevantni skoraj izključno ti dve platformi.

Takšna koncentracija moči pomeni, da sta Google in Apple osrednja posrednika med uporabnikom in napravo. Določata način nameščanja programske opreme, pravila za aplikacije, varnostne mehanizme in meje dostopnih funkcionalnosti. Čeprav tak model prinaša prednosti, kot sta večja zanesljivost in bolj enoten varnostni okvir, hkrati omejuje nadzor uporabnika nad

napravo, zlasti pri prilagajanju sistema in izbiri storitev.

## 2.2 Sistem iOS

Sistem iOS je zasnovan kot strogo nadzorovan sistem. Uporabnik aplikacije praviloma namešča prek uradne Applove trgovine z aplikacijami (App Store), sistem pa temelji na modelu podpisovanja kode in preverjanja celovitosti [11, 12]. Tak pristop prinaša enotno uporabniško izkušnjo, razmeroma predvidljiv varnostni model in dolgo programsko podporo za naprave. Hkrati pomeni tudi, da ima uporabnik omejen vpliv na to, kaj se lahko izvaja na napravi in kakšne posege v sistem je sploh mogoče izvesti.

Z vidika diplomske naloge je bistveno predvsem to, da iOS kot platforma ne omogoča resnične možnosti "popolnoma odprtokodnega telefona". Uporabnik niti teoretično nima možnosti zamenjati ključnih komponent sistema z odprtokodnimi alternativami. iOS je zato dober kontrastni primer, saj pokaže, kako je videti močno centraliziran mobilni ekosistem, kjer nadzor nad napravo ostaja predvsem v rokah proizvajalca [12].

## 2.3 Sistem Android

Android se pogosto predstavlja kot odprtokoden sistem, saj temelji na Androidovem odprtokodnem projektu (angl. *Android Open Source Project*, AOSP) [6]. To je pomembna razlika v primerjavi z iOS, ker projekt AOSP omogoča javno dostopno izvorno kodo za velik del platforme in teoretično dopušča razvoj alternativnih distribucij.

V praksi pa tipična naprava z Androidom ni enaka projektu AOSP. Večina naprav vsebuje:

- prilagoditve proizvajalca originalne opreme (angl. *original equipment manufacturer*, OEM), ki vključujejo dodatne sistemske aplikacije;
- zaprtokodne gonilnike in strojno programsko opremo za posamezne komponente, ki so del sklada dobavitelja [5, 7];

- Googlove mobilne storitve (angl. *Google Mobile Services*, GMS), ki niso del projekta AOSP in so na voljo le prek licence; pomemben del teh storitev predstavlja GMS Core oziroma Google Play Services [7, 3].

Posledica je, da je Android kot ekosistem hibriden. Odprtokodna osnova se v praksi nadgradi z več lastniškimi plastmi. To je posebej očitno pri strojni podpori. Gonilniki za grafični pospeševalnik, kamero, mobilni modem in druge ključne komponente so pogosto zaprti ter vezani na konkretno jedro in različico sistema [5, 7]. Zato posodobitve niso zgolj vprašanje nove različice Androida, temveč tudi vprašanje združljivosti celotnega sklada gonilnikov in strojne programske opreme. Del tega problema Android naslavlja z modularizacijo sistemskih komponent, kjer je del posodobitev mogoče dostaviti ločeno od celotnega sistema [8].

Takšna arhitektura vodi tudi v fragmentacijo. Različni proizvajalci vzdržujejo lastne različice sistema in jedra, kar poveča stroške vzdrževanja ter lahko negativno vpliva na hitrost in trajanje zagotavljanja posodobitev [1, 43]. To je eden izmed razlogov, zakaj naprave dobijo le omejeno število večjih posodobitev, čeprav je strojna oprema še vedno dovolj zmogljiva. Kot minimalen referenčni okvir lahko uporabimo tudi zahteve programa za priporočene poslovne naprave, ki za vključene naprave predvideva podporo trenutni izdaji in vsaj eni večji nadgradnji operacijskega sistema [2].

## 2.4 Akterji mobilnega ekosistema

Za razumevanje omejitev odprtokodnih mobilnih sistemov je ključno upoštevati, da telefon ni le operacijski sistem z aplikacijami. Pri tipični napravi sodeluje več akterjev:

- **proizvajalec operacijskega sistema oziroma ekosistema** (Apple ali Google, posredno tudi skupnost AOSP);
- **izdelovalec originalne opreme (OEM)** (npr. OnePlus), ki sistem prilagodi konkretni napravi;

- **proizvajalec sistema na čipu (angl. *system on a chip*, SoC)** (npr. Qualcomm), ki zagotovi gonilnike in strojno programsko opremo;
- **operaterji**, ki v določenih državah vplivajo tudi na certificiranje in omejitve;
- **razvijalci aplikacij**, ki se zanašajo na lastniške programske vmesnike (angl. *application programming interface*, API) in storitve, npr. potisna obvestila ali SafetyNet/Play Integrity [4, 3].

Poseben primer je modemski podsistem. Ta je pogosto ločen procesor ali podsistem s svojo strojno programsko opremo, ki je praviloma lastniška [30, 16]. Zaradi varnosti in regulative je razumljivo, zakaj je ta del izoliran, vendar z vidika odprtokodnosti predstavlja strukturno omejitev. Uporabnik njegove strojne programske opreme praviloma ne more pregledati ali zamenjati. Tudi če zamenjamo operacijski sistem in aplikacije, modemski podsistem ostaja ključen zaprt del naprave.

## 2.5 Alternativni Android ROM-i

Alternativni ROM-i za Android so eden glavnih načinov, kako tehnično bolj podkovani uporabniki povečajo nadzor nad napravo. Najbolj znan primer je LineageOS[1], poleg njega pa obstajajo tudi projekti, ki dajejo večji poudarek zasebnosti ali zmanjšanju odvisnosti od Googlovih storitev, kot sta sistema /e/OS[2] in GrapheneOS[3].

V primerjavi s tovarniškimi sistemi alternativni ROM-i navadno vključujejo manj prednaložene programske opreme, preglednejše nastavitve zasebnosti in več možnosti prilagajanja sistema. Pri tovarniških sistemih so dodatne aplikacije lahko nameščene že v izhodišču, čeprav jih uporabnik ne potrebuje in jih v nekaterih primerih tudi ne more preprosto odstraniti. Za takšno odvečno prednaloženo programsko opremo se uporablja tudi izraz *bloatware*.

[1] `https://lineageos.org/`
[2] `https://e.foundation/e-os/`
[3] `https://grapheneos.org/`

Alternativni ROM-i lahko ponudijo:

- manj prednaloženih sistemskih aplikacij;
- bolj transparentne nastavitve zasebnosti;
- več možnosti prilagajanja, npr. skrbniški dostop, drugačno obnovitveno okolje ter večji nadzor nad sistemskimi aplikacijami;
- pri nekaterih napravah tudi daljšo podporo, odvisno od aktivnosti skupnosti in vzdrževanja [15].

Ključna omejitev alternativnih ROM-ov je, da ostajajo vezani na isti temeljni problem kot tovarniški Android. Še vedno so odvisni od zaprtokodnih gonilnikov in strojne programske opreme, ki jo zagotavlja proizvajalec strojne opreme. Poleg tega se v zadnjih letih povečuje odvisnost aplikacij od lastniških storitev in mehanizmov preverjanja celovitosti, kot je Play Integrity, kar lahko na sistemih brez Googlovih storitev povzroča težave pri aplikacijah z višjimi varnostnimi zahtevami, na primer pri plačilih in bančnih aplikacijah [3, 25]. Pri zaščiti vsebin se uporabljajo tudi mehanizmi upravljanja digitalnih pravic (DRM; angl. *Digital Rights Management*), kot je Widevine, kjer lahko razlike v varnostnih ravneh vplivajo na razpoložljive funkcionalnosti oziroma kakovost predvajanja [24, 74, 13, 14].

## 2.6 Mobilni Linux: postmarketOS, Ubuntu Touch in sorodni projekti

Mobilni Linux skuša na telefon prenesti bolj klasičen model računalniškega sistema. V ospredju so standarden uporabniški prostor operacijskega sistema GNU/Linux, paketni upravljalnik, terminal in večja svoboda prilagajanja. Med bolj aktivnimi projekti sta postmarketOS[4] in Ubuntu Touch[5].

[4] `https://postmarketos.org/`
[5] `https://ubports.com/`

Projekt postmarketOS gradi na distribuciji Alpine Linux in je izrazito usmerjen v dolgoročno vzdržljivost ter ponovno uporabnost naprav [51]. S tem skuša zmanjšati odvisnost od specifičnih različic Androida in se, kjer je to izvedljivo, približati uporabi jedra glavne razvojne veje.

Ubuntu Touch razvija skupnost UBports in skuša ponuditi bolj uporabniku prijazen mobilni sistem z lastnim grafičnim vmesnikom. Pomemben del njegove zasnove je koncept konvergence, po katerem lahko ista naprava deluje kot telefon in kot namizni računalnik [71, 70]. Cilj projekta je ponuditi alternativo Androidu, ki ni vezana na Googlov ekosistem in pri kateri je večina programske opreme odprtokodna.

Kljub tem ambicijam mobilni Linux v praksi ostaja močno odvisen od strojne podpore. Pri številnih napravah je za delovanje še vedno treba uporabiti združljivostne ali premostitvene sloje za obstoječe Androidove komponente, na primer libhybris ali Halium [29, 36, 62]. To pomeni, da prehod na uporabniški prostor sistema GNU/Linux ne odpravi samodejno vseh zaprtih delov sistema, temveč pogosto vodi v hibridno rešitev. Prav zato je mobilni Linux dober primer napetosti med odprtostjo in praktično uporabnostjo: bolj odprt uporabniški prostor še ne pomeni tudi popolnoma odprte ali povsem zrele mobilne platforme.

## 2.7 Namenske Linux naprave

Poleg prenosov na obstoječe telefone Android obstajajo tudi namenske naprave, namenjene uporabnikom, ki želijo odprtokodne alternative, na primer PinePhone in Librem 5 [49, 50, 55, 58]. Te naprave skušajo položaj izboljšati z izbiro strojne opreme, ki ima boljšo podporo v jedru Linuxa ali bolj odprto dokumentacijo.

Prednost namenskih Linux telefonov je bolj konsistenten programski sklad, ki ga sestavljajo jedro, uporabniški prostor in uporabniški vmesnik, ter dejstvo, da skupnost razvoj usmerja na manjše število naprav. Slabost pa je, da se takšne naprave težko kosajo z najboljšimi telefoni s sistemom Android ali

iOS glede zmogljivosti, kakovosti kamere in porabe energije. Zato ostajajo predvsem v niši, kar vpliva tudi na velikost skupnosti in hitrost razvoja.

## 2.8 Povzetek

Pregled področja pokaže, da mobilni ekosistem danes skoraj v celoti obvladuje duopol Android/iOS, v katerem je nadzor uporabnika nad napravo v veliki meri omejen z zaprtimi ekosistemi, lastniškimi komponentami in poslovnimi modeli platform. Android sicer temelji na odprtokodni osnovi, vendar tipične naprave vključujejo več zaprtih slojev, ki otežujejo dolgoročno vzdrževanje in razvoj alternativnih sistemov. Alternativni ROM-i za Android lahko deloma povečajo nadzor nad sistemom, vendar ostajajo odvisni od istih gonilnikov in strojne programske opreme. Mobilne distribucije sistema GNU/Linux in namenske naprave z GNU/Linuxom predstavljajo ambicioznejšo alternativo, vendar njihovo praktično uporabnost še naprej omejujeta strojna podpora in aplikacijski ekosistem. To predstavlja izhodišče za naslednje poglavje, v katerem opredelimo teoretične pojme in tehnične sloje, potrebne za nadaljnjo analizo.

# Poglavje 3

# Teoretična izhodišča

V prejšnjem poglavju smo pokazali, da je tudi pri sistemih z odprtokodno osnovo velik del mobilnega ekosistema še vedno odvisen od zaprtih komponent. Da bi lahko kasneje ocenili, v kolikšni meri je popolnoma odprtokoden telefon v praksi izvedljiv, moramo najprej opredeliti osnovne pojme, sloje sistema in tipične omejitve. Pri telefonih se pogosto srečamo s položajem, v katerem je del kode odprtokoden, ključne funkcionalnosti pa so dobavljene v obliki binarnih dodatkov (angl. *binary blobs*). Odprtost nato povežemo še z varnostjo, zasebnostjo, vprašanjem stranskih vrat (angl. *backdoor*) in vzdrževanjem naprav.

## 3.1 Odprtokodna in prosta programska oprema

Pojma odprtokodna programska oprema (angl. *open source software*) in prosta programska oprema (angl. *free software*) se pogosto uporabljata kot sopomenki, vendar izhajata iz različnih poudarkov. Odprtokodnost po kriterijih Iniciative za odprto kodo (angl. *Open Source Initiative*, OSI) med drugim zahteva dostop do izvorne kode, možnost sprememb in redistribucije ter nediskriminatorne licence [48]. Organizacija za prosto programsko opremo (angl. *Free Software Foundation*, FSF) pa v ospredje postavlja uporabniške svoboščine: program je prost, če omogoča (1) poganjanje za poljuben na-

men, (2) preučevanje delovanja, (3) prilagajanje ter (4) deljenje kopij, tudi spremenjenih [20]. V nadaljevanju zato uporabljamo tudi širši izraz prosta in odprtokodna programska oprema (FOSS; angl. *Free and Open Source Software*), kadar želimo zajeti oba vidika.

Pri ocenjevanju "odprtosti telefona" je pomembno, da ločimo licenco od samega dejstva, da je koda javno dostopna. V praksi se pojavljajo tudi projekti, kjer je izvorna koda objavljena, vendar licenca ne dopušča redistribucije ali sprememb; za tak model se uporablja izraz dostopna izvorna koda (angl. *source-available*). Za kriterije v tej diplomski nalogi zato ne zadošča zgolj javna objava kode, temveč tudi pravice, ki jih določajo licence, ter možnost dejanske uporabe spremenjenih različic, na primer namestitve na napravo.

### 3.1.1 Licence in posledice za mobilne sisteme

Licence določajo pogoje uporabe, spreminjanja in redistribucije izvorne kode ter izpeljanih del. V grobem jih lahko razdelimo na:

- **permisivne licence** (npr. MIT [68], BSD [66], Apache 2.0 [10]), ki omogočajo vključitev kode tudi v lastniške izdelke, ter

- **licence z zahtevami deljenja izboljšav** (angl. *copyleft*, npr. GPLv2 [23]), ki ob redistribuciji pod določenimi pogoji zahtevajo, da so izpeljana dela na voljo pod enako ali združljivo licenco.

Za mobilne naprave je posebej pomembno jedro Linux, ki je licencirano pod GPLv2 [23]. To proizvajalcem pri distribuciji naprav nalaga obveznosti glede objave izvorne kode sprememb jedra, vendar še ne pomeni, da je odprt celoten programski sklad. V praksi lahko ostanejo zaprti posamezni gonilniki, strojna programska oprema ter velik del uporabniškega prostora, na primer sistemske aplikacije ali dobaviteljske knjižnice.

## 3.2 Arhitektura sistema: od zagona do uporabniškega prostora

Pametni telefon je sestavljen iz več podsistemov. Poleg glavnega procesorja (SoC) ima naprava pogosto ločene mikrokrmilnike ali koprocesorje za modem, Wi-Fi/Bluetooth, kamero in senzorje, pogosto pa tudi varnostne podsisteme, kot je zaupanja vredno izvajalno okolje (TEE; angl. *Trusted Execution Environment*). Ti deli praviloma uporabljajo lastno strojno programsko opremo in so pomemben del vprašanja zaupanja, ker so za uporabnika večinoma nepregledni.

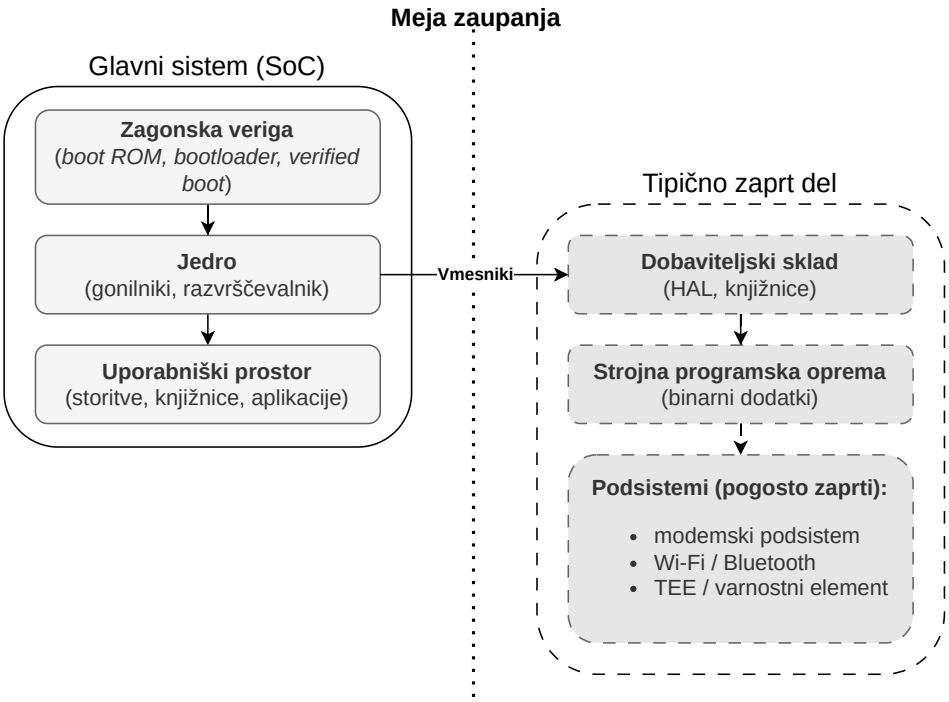
Na ravni glavnega sistema ločimo:

- zagonsko verigo (angl. *bootchain*), ki določa, kaj se sme zagnati, vključno z zagonskim nalagalnikom in preverjenim zagonom (angl. *verified boot*),
- jedro z gonilniki in
- uporabniški prostor, v katerem tečejo sistemske storitve, knjižnice in aplikacije.

Slika 3.1 povzema sloje sistema in tipične zaprte komponente, ki predstavljajo glavno omejitev pri popolnoma odprtokodnem telefonu.

### 3.2.1 Linux: glavna razvojna veja jedra in vzdrževanje

Sklad GNU/Linux je družina sistemov, ki delijo isto jedro, nad njim pa lahko teče različna programska oprema uporabniškega prostora. Za dolgoročno vzdrževanje naprav je ključno, ali je podpora za strojno opremo vključena v glavno razvojno vejo jedra ali pa je vezana na proizvajalčevo prilagojeno različico jedra [67]. Če je podpora del glavne veje, lahko naprava dobiva varnostne popravke skupaj z ostalim ekosistemom. Če pa je odvisna od zaprtih ali slabo vzdrževanih sprememb dobavitelja, je življenjska doba programske podpore krajša.




Slika 3.1: Poenostavljena arhitektura telefona z vidika slojev sistema in tipičnih zaprtih komponent.

### 3.2.2 Android: AOSP in ločitev dobaviteljskega sklada

Android temelji na jedru Linux, vendar ima specifičen uporabniški prostor in nabor sistemskih storitev, ki skupaj tvorijo platformo [6]. Za odprtost je pomembna meja med generičnim sistemom in strojno specifičnimi deli. Projekt Treble je uvedel bolj jasno ločitev na particije in vmesnike, na primer med particijama `system` in `vendor`, s čimer sistem teoretično omogoča posodabljanje neodvisno od dobaviteljskega sklada [5, 7]. Kljub temu dobaviteljski del pogosto vsebuje zaprt nabor knjižnic, implementacije abstrakcijskega sloja strojne opreme (HAL), gonilnike in strojno programsko opremo, brez katerih naprava ne deluje.

## 3.3 Gonilniki, strojna programska oprema in binarni dodatki

Pri telefonih s sistemom Android se pogosto srečamo s položajem, v katerem je del kode odprtokoden, ključne funkcionalnosti pa so na voljo le v obliki binarnih dodatkov. Pomembno je ločiti med gonilniki (angl. *drivers*) in strojno programsko opremo (angl. *firmware*).

Gonilniki predstavljajo kodo, ki teče v jedru ali uporabniškem prostoru in omogoča komunikacijo s strojno opremo.

Strojna programska oprema predstavlja kodo, ki teče na sami komponenti, na primer na čipu Wi-Fi, namenskem procesorju za obdelavo digitalnih signalov (angl. *digital signal processor*, DSP) ali modemu, sistem pa jo zgolj naloži ali upravlja.

Tudi če so gonilniki odprtokodni, je strojna programska oprema lahko popolnoma zaprta, brez možnosti revizije ali popravkov. To je ena glavnih tehničnih ovir za telefon, pri katerem bi bil celoten programski sklad pregleden in zamenljiv. Najbolj problematični so podsistemi z visokimi privilegiji ali neposrednim vplivom na komunikacijo, na primer modemski podsistem in brezžični vmesniki. Raziskave kažejo, da se ranljivosti in napadi pogosto selijo prav v zaprt dobaviteljski sklad in strojno programsko opremo, kjer je preglednost manjša, orodja za analizo pa omejena [30, 16].

## 3.4 Odprtost, varnost in zasebnost

Odprta koda lahko prispeva k varnosti, ker omogoča neodvisen pregled in hitrejše odkrivanje napak. Vendar odprtost sama po sebi ni garancija; varnost je predvsem rezultat procesa, torej revizij, testiranja, odzivnosti na ranljivosti in nadzora nad tem, kaj se na napravi dejansko izvaja. Pri telefonih zato poleg licence ocenjujemo tudi oskrbovalno verigo, mehanizme posodabljanja in stopnjo zaprtih komponent.

### 3.4.1 Stranska vrata in oskrbovalna veriga

Stranska vrata (angl. *backdoor*) razumemo kot namerno dodano ali prikrito funkcionalnost, ki omogoča dostop mimo običajnih varnostnih mehanizmov. Pri zaprtih komponentah je problem očiten: uporabnik in neodvisni raziskovalci težko preverijo, ali takšna funkcionalnost obstaja. Tudi pri odprtokodni programski opremi pa ostajata pomembna napadna vektorja oskrbovalna veriga in orodja za izgradnjo. Klasičen primer je Thompsonov opis, kako lahko kompromitiran prevajalnik vstavi stranska vrata v program tudi takrat, ko je izvorna koda navidezno "čista" [69]. Za oceno zaupanja je zato smiselno gledati širše od same izvorne kode.

Pri telefonih se pomisleki o stranskih vratih najpogosteje vežejo na:

- zaprt dobaviteljski sklad na komunikacijskih čipih (modem, Wi-Fi/Bluetooth),
- privilegirane varnostne podsisteme (npr. TEE in varnostni element) in
- lastniške sistemske storitve s širokimi dovoljenji (npr. telemetrija).

### 3.4.2 Celovitost zagona in ponovljive gradnje

Moderni telefoni pogosto uporabljajo mehanizme preverjenega zagona, pri katerih je zagonska veriga kriptografsko vezana na podpisane sistemske slike. To izboljša zaščito pred zlonamernimi spremembami, hkrati pa lahko omeji lastnika naprave, če je zagonski nalagalnik zaklenjen in proizvajalec ne omogoča nalaganja alternativnih podpisov [9]. Za sisteme, ki ciljajo na odprtost, je zato ključno ravnotežje: visoka stopnja celovitosti zagonske verige, hkrati pa možnost, da lastnik na napravo naloži in s svojimi ključi podpiše lastno različico sistema.

Dodaten sloj zaupanja predstavljajo ponovljive gradnje (angl. *reproducible builds*), pri katerih lahko različni neodvisni razvijalci iz iste izvorne kode izdelajo identične binarne pakete [61]. S tem zmanjšamo tveganje, da bi bila uradna izdaja kompromitirana na poti od kode do binarne slike. Pri telefonih je to posebej relevantno, ker uporabniki praviloma nameščajo vnaprej

zgrajene sistemske slike, ne pa da bi sistem prevajali sami.

### 3.4.3 Zasebnost kot posledica arhitekture

Zasebnost pri telefonih ni samo vprašanje aplikacij, temveč tudi sistemske arhitekture. Če naprava temelji na zaprtih storitvah in naborih programskih vmesnikov, uporabnik težko oceni, kateri podatki se pošiljajo in kdaj. Alternativni sistemi lahko zmanjšajo odvisnost od centraliziranih storitev, vendar se pri tem pogosto pojavi kompromis pri združljivosti aplikacij, zlasti pri aplikacijah z višjimi varnostnimi zahtevami, ki uporabljajo mehanizme preverjanja celovitosti [3].

## 3.5 Vzdrževanje in življenjski cikel naprav

Pri oceni odprtokodnega telefona je praktična uporabnost tesno povezana z vzdrževanjem. Če naprava ne dobiva varnostnih popravkov, odprtost hitro postane zgolj formalna lastnost. Analize ekosistema Android kažejo velike razlike v dolžini podpore in zamudah pri varnostnih posodobitvah med proizvajalci [1]. V mobilnem Linux svetu se ta problem pogosto rešuje z uporabo glavne razvojne veje jedra in standardnega uporabniškega prostora, vendar je cena slabša strojna podpora in manj zrel ekosistem aplikacij.

V naslednjih poglavjih bomo na podlagi teh izhodišč postavili merljive kriterije, kaj v našem kontekstu pomeni “popolnoma odprtokoden telefon”, nato pa bomo z njimi analizirali izbrane obstoječe rešitve.

# Poglavje 4

# Model "popolnoma odprtokodnega telefona"

V prejšnjih poglavjih smo pokazali, da mobilni telefon ni le operacijski sistem z aplikacijami, temveč preplet več slojev programske opreme, strojne opreme, dobaviteljskih komponent in mehanizmov zagona. Zato pojem "popolnoma odprtokoden telefon" v praksi ni samoumeven: pri pametnem telefonu odprtost ni odvisna le od licence operacijskega sistema, temveč tudi od gonilnikov, strojne programske opreme, sistemskih storitev, načina nameščanja aplikacij in možnosti dolgoročnega vzdrževanja.

V tem poglavju najprej opredelimo idealni referenčni model popolnoma odprtokodnega telefona. Nato določimo kriterije, s katerimi bomo konkretne rešitve primerjali glede na dve ločeni dimenziji: **odprtost** in **praktična uporabnost**. Odprtost sama po sebi še ne pomeni, da je naprava uporabna kot vsakdanji telefon, visoka uporabnost pa še ne pomeni, da rešitev uporabniku omogoča dejanski nadzor nad sistemom. Zato moramo obe dimenziji obravnavati ločeno.

## 4.1 Idealni referenčni model

Če pojem "popolnoma odprtokoden" razumemo dobesedno, bi to pomenilo, da je vsa programska oprema, ki se izvaja na napravi, na voljo kot prosta in odprtokodna programska oprema, z možnostjo pregleda, sprememb in redistribucije [48, 20]. To ne vključuje le uporabniškega prostora in jedra, temveč tudi gonilnike, zagonsko verigo, sistemske storitve in strojno programsko opremo pomožnih podsistemov.

Tak idealni telefon bi moral hkrati izpolnjevati več pogojev. Celoten sistem bi moral biti zgradljiv iz izvorne kode, ključne komponente pa neodvisno preverljive, po možnosti tudi s ponovljivimi gradnjami [61]. Lastnik naprave bi moral imeti možnost na dokumentiran in ponovljiv način namestiti lasten sistem, ne da bi se moral pri tem odpovedati preverjenemu zagonu ali postati popolnoma odvisen od proizvajalca. Pomembne funkcije telefona bi morale ostati uporabne tudi brez zaprtih, centraliziranih ali proizvajalčevo vezanih storitev.

V kontekstu sodobnih pametnih telefonov je tak ideal težko dosegljiv. V praksi se skoraj vedno pojavijo vsaj tri pomembne omejitve: (1) zaprt modemski podsistem, (2) zaprta strojna programska oprema za Wi-Fi/Bluetooth, kamero, grafični procesor in druge podsisteme ter (3) mehanizmi preverjenega zagona, ki lahko omejijo nalaganje lastnih gradenj, če lastnik nima nadzora nad ključi oziroma je zagonski nalagalnik zaklenjen [9].

V nadaljevanju zato ta idealni telefon obravnavamo kot referenčno točko. Obstoječih rešitev ne ocenjujemo binarno, temveč glede na to, kako blizu so temu idealu v dveh dimenzijah: (1) **odprtost** in (2) **praktična uporabnost**.

## 4.2 Slojni pogled na odprtost telefona

Za ocenjevanje odprtosti potrebujemo slojni model, ki je skladen z arhitekturo iz slike 3.1. V nadaljevanju uporabljamo naslednje sloje:

1. **Zagonska veriga**: od začetnega zagona do nalaganja jedra in upo-

rabniškega prostora.

2. **Jedro in gonilniki**: jedro Linux ter gonilniki v jedru oziroma uporabniškem prostoru.

3. **Strojna programska oprema**: programska oprema pomožnih podsistemov (modem, Wi-Fi/Bluetooth, grafični procesor, kamera, senzorji ipd.).

4. **Sistemske storitve in vmesni sloj** (angl. *middleware*): npr. sistemske storitve in abstrakcijska plast strojne opreme v Androidu oziroma klasične storitve GNU/Linux v uporabniškem prostoru.

5. **Privzete aplikacije in distribucija aplikacij**: osnovni nabor aplikacij ter način nameščanja in posodabljanja dodatnih aplikacij.

Tak slojni pogled je primeren za ocenjevanje odprtosti, ker omogoča ločeno obravnavo različnih vrst zaprtosti. Rešitev je lahko na primer razmeroma odprta na ravni uporabniškega prostora, hkrati pa močno omejena na ravni strojne programske opreme. Prav zato odprtosti ne obravnavamo kot ene same lastnosti, temveč kot profil po posameznih slojih sistema.

## 4.3 Kriteriji odprtosti po slojih

V tem razdelku postavimo kriterije, s katerimi bomo kasneje primerjali konkretne rešitve. Kriteriji so oblikovani tako, da so čim bolj preverljivi: zanima nas, ali je nekaj javno objavljeno, neodvisno zgradljivo, dokumentirano, dolgoročno vzdrževano ali zamenljivo.

### 4.3.1 Zagonska veriga

Pri zagonski verigi nas zanima predvsem, ali ima lastnik naprave realno možnost namestiti lasten sistem, ne da bi bil pri tem popolnoma odvisen od proizvajalca. Pri tem ni pomembna le možnost odklenitve naprave, temveč

tudi vprašanje, ali je mogoče ohraniti preverjen zagon in razumne varnostne lastnosti sistema.

Kriteriji:

- **Odklep zagonskega nalagalnika**: ali je uradno podprt in dokumentiran ter ali ne zahteva nepovratnih korakov, ki uporabnika trajno omejijo.

- **Preverjen zagon z uporabniškimi ključi**: ali sistem omogoča preverjen zagon, pri čemer lahko naprava zaupa tudi podpisom, ki jih upravlja lastnik [9].

- **Dokumentiranost namestitve**: ali je postopek namestitve alternativnega sistema javno opisan in reproducibilen.

### 4.3.2 Jedro in gonilniki

Pri jedru in gonilnikih je ključno vprašanje dolgoročno vzdrževanje. Če je podpora za napravo vključena v glavno razvojno vejo jedra Linux, je možnost dolgotrajnih varnostnih posodobitev večja [67]. Nasprotno pa dobaviteljske izpeljanke in zaprtokodni gonilniki povečujejo odvisnost od proizvajalca ter otežujejo neodvisno vzdrževanje.

Kriteriji:

- **Podpora v glavni veji**: ali so ključni gonilniki vključeni v uradno jedro ali pa gre za dobaviteljsko izpeljanko.

- **Odprtost gonilnikov**: ali so gonilniki odprtokodni ali pa so prisotni binarni dodatki v jedru oziroma uporabniškem prostoru.

- **Vzdrževalni signal**: ali je vidna aktivnost posodobitev, varnostnih popravkov in vključevanja sprememb v izvorni projekt.

### 4.3.3 Strojna programska oprema

Strojna programska oprema je največja ovira za strogo odprt telefon. Pri sodobnih pametnih telefonih je velik del te programske opreme tesno vezan na dobavitelje čipov, strojno zasnovo naprave in regulativne zahteve. Popolna odprtost zato pogosto ni realno dosegljiva.

V takem primeru je pomembno predvsem vprašanje, katere komponente ostajajo zaprte in kako pomembne so za delovanje telefona. Pri ocenjevanju zato ne upoštevamo le tega, ali je posamezna komponenta odprta, temveč tudi, kako pomembna je za delovanje naprave in kako privilegiran položaj ima v arhitekturi sistema.

Podsistemov ne obravnavamo kot enakovrednih, temveč jih pri sintezni oceni razvrstimo v tri razrede glede na njihov vpliv na celoten sistem. V prvi razred sodijo kritični podsistemi, ki imajo širok dostop do podatkov, vplivajo na komunikacijo z zunanjim svetom ali predstavljajo temelj zaupanja v sistem; sem uvrščamo predvsem modemski podsistem, podsisteme za brezžično povezljivost ter privilegirane varnostne podsisteme, kot sta TEE in varnostni element. V drugi razred sodijo pomembni podsistemi, ki bistveno vplivajo na uporabnost in zasebnost naprave, vendar nimajo enako privilegiranega položaja, na primer kamera, grafični podsistem in GNSS. V tretji razred sodijo manj kritični podsistemi z ožjim vplivom na celoten sistem, na primer posamezni senzorji ali manj pomembni periferni elementi. Pri ocenjevanju zato ne gledamo le na prisotnost zaprtih komponent, temveč tudi na njihov razred pomembnosti, privilegiranost in obseg vpliva. Zaprta komponenta v prvem razredu lahko bistveno zniža sintezno oceno sloja, medtem ko zaprta komponenta v tretjem razredu sama po sebi še ne pomeni nizke ocene. Cilj takšne razvrstitve ni ustvariti navidezno natančnega uteženega modela, temveč transparentno pojasniti, zakaj imajo nekatere zaprte komponente pri končni presoji večjo težo kot druge.

Kriteriji:

- **Razpoložljivost izvorne kode:** ali je strojna programska oprema odprtokodna; če ni, ali obstajajo odprte alternative, četudi z omejitvami.

- **Obseg, pomembnost in privilegiji:** kateri podsistemi so zaprti, ali sodijo med kritične, pomembne ali manj kritične podsisteme, kako privilegiran položaj imajo v arhitekturi sistema in kako kritični so za delovanje naprave; večjo težo imajo kritični podsistemi, kot so modem, brezžični vmesniki ter privilegirani varnostni podsistemi, manjšo pa podsistemi z ožjim vplivom na celoten sistem.

### 4.3.4 Sistemske storitve in vmesni sloj

Pri Androidu je pomembna meja med odprtim jedrom platforme (AOSP) ter dobaviteljskimi komponentami in implementacijami abstrakcijske plasti strojne opreme, pri mobilnem GNU/Linux pa predvsem to, ali je uporabniški prostor sestavljen iz standardnih paketov distribucije ali iz posebnega, projektno vezanega sklada.

Pri tem sloju nas ne zanima neposredno, ali je sistem varen ali zaseben v absolutnem smislu, temveč predvsem to, v kolikšni meri omogoča neodvisen pregled in preverjanje. Odprt sistem je v tem kontekstu tak, pri katerem je ključne dele mogoče pregledati, po potrebi zamenjati ter vsaj deloma neodvisno preveriti, kaj se dejansko izvaja na napravi. Poudarek je torej na preverljivosti sistema in zmanjšanju potrebe po slepem zaupanju v zaprt dobaviteljski sklad.

Kriteriji:

- **Odprtost ključnih sistemskih komponent:** ali ključne sistemske storitve in vmesni sloji temeljijo pretežno na odprtokodnih in zamenljivih komponentah ali pa so za osnovno delovanje sistema bistveni zaprtokodni deli.

- **Odvisnost od lastniških storitev**: ali sistem za osnovno delovanje zahteva centralizirane, zaprtokodne storitve ali pa ima odprte alternative.

- **Preglednost izvora komponent**: ali so ključne sistemske komponente jasno dokumentirane, z javno dostopno izvorno kodo in jasno

ločitvijo med odprtimi in zaprtimi deli sklada.

- **Neodvisna preverljivost izdaj**: ali je realno mogoče neodvisno preveriti binarne izdaje, vsaj za kritične dele sistema, na primer s ponovljivimi gradnjami [61].

### 4.3.5 Privzete aplikacije in distribucija aplikacij

Na zadnjem sloju se pokaže praktična plat sistema: tudi zelo odprt sistem je za mnoge uporabnike težko uporaben, če nima preglednega in zanesljivega načina nameščanja aplikacij. V tej nalogi kot ustrezen način razumemo takšnega, ki omogoča nameščanje in posodabljanje aplikacij iz zaupanja vrednega vira brez nestandardnih postopkov ter s preverljivimi posodobitvami.

Pri tem sloju nas zanimata predvsem dve vprašanji. Prvo je, kakšen nabor aplikacij uporabnik dobi že v privzeti namestitvi sistema. Drugo pa je, ali sistem tudi po koncu uradne podpore proizvajalca še omogoča smiselno neodvisno vzdrževanje in nameščanje dodatne programske opreme.

Vprašanje telemetrije privzeto nameščenih aplikacij je za uporabnika sicer pomembno, vendar ga v tej nalogi ne vključujemo kot samostojnega ocenjevalnega kriterija. Pri zaprtokodnih aplikacijah namreč brez poglobljene dinamične analize ali postopkov obratnega inženiringa (angl. *reverse engineering*) pogosto ni mogoče zanesljivo ugotoviti, kateri podatki se prenašajo, v kakšnem obsegu in za kakšen namen. Telemetrijo zato v nadaljevanju obravnavamo le opisno, kadar je to pomembno za razpravo o posamezni rešitvi.

Kriteriji:

- **Odprtost privzeto nameščenih aplikacij:** ali so aplikacije iz privzete namestitve sistema pretežno odprtokodne ali pa med njimi prevladujejo zaprtokodne aplikacije.

- **Trgovina in repozitoriji:** ali sistem omogoča nameščanje aplikacij iz odprtih ali neodvisno vzdrževanih virov ter ali uporabnik pri nameščanju dodatne programske opreme ni povsem odvisen od enega samega lastniškega distribucijskega kanala.

- **Neodvisnost od proizvajalca:** ali uporabnik lahko vzdržuje sistem in aplikacije tudi po tem, ko proizvajalec naprave uradno zaključi podporo.

### 4.3.6 Povzetek

Za lažji pregled tabela 4.1 povzema sloje in tipične kriterije odprtosti, ki jih bomo uporabili v nadaljevanju.

| Sloj | Kaj preverjamo |
|---|---|
| Zagonska veriga | odklep zagonskega nalagalnika; preverjen zagon z uporabniškimi ključi; dokumentiran postopek namestitve |
| Jedro in gonilniki | podpora v glavni veji; odprtost gonilnikov; aktivno vzdrževanje |
| Strojna programska oprema | odprta koda ali odprte alternative; obseg zaprtih podsistemov; izolacija |
| Sistemske storitve | delež komponent FOSS; odvisnost od lastniških storitev; ponovljive gradnje |
| Aplikacije in distribucija | odprtokodne privzete aplikacije; odprti repozitoriji; neodvisnost od proizvajalca |

Tabela 4.1: Povzetek slojev in tipičnih kriterijev odprtosti v predlaganem modelu.

## 4.4 Kriteriji praktične uporabnosti

Ker je ciljna naprava telefon, mora sistem pokriti vsaj osnovne funkcionalnosti, sicer primerjava ni smiselna. Odprt sistem brez delujočih klicev je lahko zanimiv kot tehnična demonstracija, vendar ne deluje kot telefon v običajnem pomenu besede.

Praktične uporabnosti v tej nalogi ne ocenjujemo po slojih sistema, temveč

po funkcionalnih sklopih, ki skupaj določajo, ali je naprava primerna za vsakodnevno rabo. Izhodišče je minimalni nabor funkcionalnosti za vsakodnevno uporabo: naprava mora omogočati osnovno telefonijo, povezljivost, dostop do ključnih aplikacij, razumno avtonomijo ter dovolj enostavno namestitev, uporabo in vzdrževanje.

Ker se diplomska naloga ne osredotoča na povprečnega uporabnika, temveč predvsem na tehnično bolj podkovanega uporabnika z višjimi zahtevami glede zasebnosti in nadzora nad sistemom, kot del praktične uporabnosti upoštevamo tudi stopnjo prilagodljivosti sistema. Pri telefonu takšnega tipa ni pomembno le, ali funkcija deluje, temveč tudi, v kolikšni meri lahko uporabnik sistem samostojno pregleda, prilagaja in upravlja brez pretirane odvisnosti od proizvajalca ali nestandardnih obhodov.

V nadaljevanju uporabljamo naslednje sklope:

- **Osnovne funkcije telefona**: klici, SMS, mobilni podatki in stabilnost povezave.
- **Ključna strojna podpora**: kamera, GPS, Bluetooth, Wi-Fi, zvok in senzorji; po potrebi tudi NFC.
- **Avtonomija in zmogljivost**: poraba energije v mirovanju in pri rabi, segrevanje ter odzivnost sistema.
- **Ekosistem aplikacij**: ali so za minimalni nabor vsakodnevnih nalog na voljo ustrezne aplikacije ali spletne alternative.
- **Posodobitve in vzdrževanje**: kako pogosto prihajajo varnostne posodobitve in kako enostavno jih je namestiti.
- **Prilagodljivost in uporabniški nadzor**: v kolikšni meri lahko uporabnik samostojno prilagaja sistem, upravlja nameščeno programsko opremo, uporablja naprednejša orodja (npr. terminal ali paketni upravljalnik) ter nadzira procese, storitve in dovoljenja v ozadju.

Pri teh sklopih nas zanima predvsem, ali posamezne funkcije delujejo dovolj zanesljivo, da je naprava primerna za vsakodnevno rabo. Pri zadnjem sklopu pa nas dodatno zanima, ali sistem uporabniku omogoča razumno stopnjo dejanskega nadzora nad napravo, saj je to pri odprtokodnih mobilnih rešitvah pomemben del njihove praktične vrednosti.

Pri ekosistemu aplikacij v tej nalogi preverjamo, ali so za vnaprej opredeljen minimalni nabor vsakodnevnih nalog na voljo ustrezne aplikacije ali spletne alternative. Kot minimalne kategorije upoštevamo komunikacijo, navigacijo, spletni brskalnik, e-pošto, kamero in galerijo, predvajanje medijev ter osnovne aplikacije za avtentikacijo, npr. dvofaktorsko avtentikacijo. Posebej obravnavamo tudi aplikacije z višjimi varnostnimi zahtevami, kot so bančne aplikacije in storitve mobilnega plačevanja, saj pogosto uporabljajo mehanizme preverjanja celovitosti. Takšne aplikacije so zato dober pokazatelj praktičnih omejitev posamezne rešitve [3].

## 4.5 Način ocenjevanja

Za primerjavo obstoječih projektov in za študijo primera potrebujemo enoten način ocenjevanja. Posamezne rešitve ocenjujemo v dveh ločenih dimenzijah: odprtosti in praktični uporabnosti. Pri tem ne uporabljamo dveh ločenih lestvic z enakimi vrednostmi, temveč eno samo tristopenjsko lestvico z ocenami 0, 1 in 2, ki jo razlagamo glede na posamezni kriterij.

Ne ocenjujemo projekta v splošnem, temveč vedno konkretno rešitev na konkretni podprti napravi. Tako na primer ne ocenjujemo "GrapheneOS na splošno", temveč "GrapheneOS na uradno podprti napravi". S tem se izognemo mešanju lastnosti programske rešitve z lastnostmi strojne opreme.

Odprtost ocenjujemo po slojih iz sekcije 4.2 in po kriterijih iz sekcije 4.3, praktično uporabnost pa po sklopih iz sekcije 4.4. V obeh dimenzijah uporabljamo isto abstraktno lestvico, ker posamezni kriteriji odprtosti ali uporabnosti ne merijo vedno neposredno, temveč pogosto merijo lastnosti, iz katerih lahko na odprtost oziroma uporabnost sklepamo. To velja na primer za

dokumentiranost namestitve, neodvisnost od lastniških storitev, ponovljive gradnje, vzdrževalni signal ali odprtost ključnih sistemskih komponent.

Model ne ocenjuje neposredno varnosti, zasebnosti ali dejanske zaupnosti sistema. Te vidike obravnavamo kot pomembne posledice odprtosti in preverljivosti, vendar jih zaradi omejenega obsega naloge ne vključujemo kot samostojne dimenzije ocenjevanja. V model odprtosti zato vključujemo predvsem tiste lastnosti, ki uporabniku in neodvisni skupnosti omogočajo pregled, zamenljivost komponent, preverljivost izdaj ter dolgoročno vzdrževanje sistema.

Pri sintezni oceni posameznega sloja oziroma sklopa ne obravnavamo vseh kriterijev in podsistemov kot enako pomembnih. Formalnega uteževanja ne uporabljamo, ker bi tak pristop ustvaril vtis večje natančnosti, kot jo je pri tovrstni primerjavi mogoče utemeljiti. Kljub temu pri razlagi rezultatov upoštevamo hierarhijo pomembnosti. Večjo težo imajo komponente, ki so nujne za osnovno delovanje telefona, imajo širok dostop do podatkov ali zasedajo privilegiran položaj v arhitekturi sistema. Pri odprtosti to pomeni, da imajo večjo težo zagonska veriga, modem, brezžični vmesniki, privilegirani varnostni podsistemi ter ključni gonilniki in sistemske storitve kot pa posamezne manj kritične komponente. Pri praktični uporabnosti pa imajo večjo težo osnovne funkcije telefona, mobilni podatki, stabilnost sistema in dostop do ključnih aplikacij kot pa manj pogosto uporabljene ali nadomestljive funkcije.

Skupna ocenjevalna lestvica je naslednja:

- **0 – nizka ocena:** obravnavani kriterij je izpolnjen slabo ali pa sploh ni izpolnjen. Pri odprtosti to pomeni, da rešitev ostaja pretežno odvisna od zaprtih, nezamenljivih ali slabo preverljivih komponent. Pri praktični uporabnosti pa pomeni, da funkcionalnost ne deluje ali pa je za vsakdanjo rabo preveč nezanesljiva.

- **1 – srednja ocena:** obravnavani kriterij je izpolnjen delno. Rešitev na tem mestu pokaže določen napredek, vendar ostajajo pomembne omejitve, kompromisi ali odvisnosti.

- **2 – visoka ocena:** obravnavani kriterij je izpolnjen dobro. Pri odprtosti to pomeni, da je rešitev pretežno odprta, preverljiva in zamenljiva. Pri praktični uporabnosti pa pomeni, da funkcionalnost deluje stabilno in brez posebnih omejitev v tipičnih scenarijih uporabe.

Ocena posameznega sloja oziroma sklopa je sintezna ocena na podlagi vnaprej določenih kriterijev, pri čemer pri razlagi upoštevamo tudi relativno pomembnost posameznih podsistemov in funkcij. Skupnega uteženega rezultata ne računamo. Rezultate zato predstavimo v tabelah in grafičnih povzetkih, ločeno za odprtost in praktično uporabnost.

Pri analizi obstoječih rešitev ocene temeljijo na javno dostopni dokumentaciji projektov, seznamih podprtih funkcionalnosti, znanih omejitvah in uporabniških poročilih. V empiričnem delu iste sklope ocenimo še na podlagi lastnega testiranja na izbrani napravi. Ocene v primerjalni analizi so zato nujno bolj okvirne kot ocene v študiji primera, kjer lahko funkcionalnosti preverimo neposredno.

## 4.6 Povzetek

V tem poglavju smo idealni popolnoma odprtokodni telefon opredelili kot referenčno točko za nadaljnjo primerjavo. Nato smo določili kriterije odprtosti po slojih sistema in kriterije praktične uporabnosti po funkcionalnih sklopih telefona. Posamezne rešitve bomo v nadaljevanju ocenjevali ločeno v obeh dimenzijah, brez združevanja v en sam skupni rezultat.

Tak pristop omogoča bolj pregledno in pošteno primerjavo. Namesto navidezno natančne končne številke dobimo profil posamezne rešitve, iz katerega je razvidno, kje so njene prednosti, kje omejitve in kateri kompromisi so posledica konkretnih tehničnih ali organizacijskih odločitev projekta.

# Poglavje 5

# Analiza trenutnega stanja odprtokodnih mobilnih rešitev

V prejšnjem poglavju smo oblikovali model, s katerim lahko posamezne mobilne rešitve primerjamo glede na dve ločeni dimenziji: **odprtost** in **praktična uporabnost**. V tem poglavju ta model uporabimo za analizo obstoječih odprtokodnih mobilnih rešitev.

Namen poglavja ni zgolj izdelati seznama projektov, temveč pokazati, kako posamezne rešitve v praksi različno razporejajo kompromise med odprtostjo, varnostjo, zrelostjo programske podpore in vsakdanjo uporabnostjo. Pri tem nas ne zanima le, ali je projekt "odprt", temveč tudi, *kateri del* sistema je odprt, kje ostajajo odvisnosti od zaprtih komponent in kako močno to vpliva na dejansko uporabo naprave.

Analizirane rešitve razdelimo v tri večje skupine: (1) alternativne ROM-e za sistem Android, (2) mobilne distribucije sistema GNU/Linux na obstoječi strojni opremi za sistem Android in (3) namenske telefone z operacijskim sistemom GNU/Linux. Takšna delitev je smiselna, ker posamezne skupine rešujejo isti problem z različnimi tehničnimi izhodišči. Alternativni ROM-i za sistem Android ohranijo največ praktične uporabnosti, mobilne distribucije sistema GNU/Linux povečajo odprtost uporabniškega prostora, namenski telefoni z operacijskim sistemom GNU/Linux pa se najbolj približajo idealu

arhitekturnega nadzora nad celotnim sistemom.

V nadaljevanju za celoten operacijski sistem uporabljamo izraz GNU/Linux, medtem ko izraz Linux uporabljamo le, kadar govorimo o jedru Linux.

V tem poglavju uporabljamo kriterije odprtosti iz sekcije 4.3 in kriterije praktične uporabnosti iz sekcije 4.4. Ker gre predvsem za sintezo javno dostopne dokumentacije projektov, seznamov podprtih naprav, znanih omejitev in razvojnih usmeritev, so ocene v tem poglavju nujno bolj okvirne kot v empiričnem delu naloge. Kljub temu takšna analiza omogoča dovolj jasen pregled nad tem, katere vrste kompromisov so značilne za posamezno družino rešitev.

## 5.1 Alternativni ROM-i za sistem Android

Alternativni ROM-i za sistem Android so danes najpomembnejša skupina odprtokodnih mobilnih rešitev za uporabnike, ki želijo visoko stopnjo praktične uporabnosti ob hkratnem zmanjšanju odvisnosti od tovarniškega sistema. Njihova ključna prednost je, da ohranijo združljivost z aplikacijskim ekosistemom sistema Android in s tem velik del vsakdanje uporabnosti telefona.

Ta skupina hkrati zelo jasno pokaže, da odprtost mobilnega operacijskega sistema ni enaka odprtosti celotne naprave. Čeprav ti projekti večinoma temeljijo na odprtokodnem jedru platforme Android, pri konkretni podpori posamezni napravi ostanejo vezani na dobaviteljske komponente, zaprtokodne implementacije abstrakcijske plasti strojne opreme, binarne dodatke in strojno programsko opremo posameznih podsistemov [6, 5, 7]. Zato so alternativni ROM-i za sistem Android najmočnejši na ravni uporabniškega prostora in sistemskih storitev, bistveno šibkejši pa na ravni gonilnikov, strojne programske opreme in celotne zagonske verige.

V tej skupini obravnavamo štiri reprezentativne primere: LineageOS[1],

[1] `https://lineageos.org/`

GrapheneOS[2], /e/OS[3] in Replicant[4]. Izbrani so zato, ker predstavljajo štiri različne strategije: široko podprt splošnonamenski ROM, varnostno usmerjen ROM za ožji nabor naprav, zasebnosti prijazno in uporabniško usmerjeno alternativo brez Googlovih storitev ter strogo svobodno rešitev za sistem Android, ki zavrača običajne pragmatične kompromise.

### 5.1.1 LineageOS

Projekt LineageOS je eden najbolj razširjenih predstavnikov alternativnih ROM-ov za sistem Android. Predstavlja se kot prost in odprtokoden operacijski sistem za različne naprave, ki temelji na platformi Android, pri čemer za uradno podprte naprave objavlja tudi natančna navodila za namestitev in gradnjo sistema [41, 37].

Glavna prednost projekta LineageOS je, da odprtokoden sistemski sloj združi z dovolj visoko stopnjo praktične uporabnosti. Uporabnik dobi sistem brez tovarniške preobleke proizvajalca, brez obvezne vezanosti na Googlove privzete aplikacije in s preglednim postopkom namestitve ter posodabljanja [39]. Za številne naprave zato LineageOS predstavlja eno najbolj realističnih možnosti, kako podaljšati življenjsko dobo telefona in hkrati pridobiti več nadzora nad sistemom [15].

LineageOS je hkrati tudi dober primer omejitev, ki jih ima skoraj vsak ROM za sistem Android. Uradni odgovori na pogosta vprašanja poudarjajo, da projekt za vzdrževane naprave redno vključuje varnostne popravke iz projekta AOSP, medtem ko so varnostni popravki za dobaviteljski del sistema še vedno vezani na izvor zaprtih komponent iz tovarniškega sistema [38]. To pomeni, da LineageOS poveča odprtost zgornjih slojev sistema, ne odpravi pa odvisnosti od proizvajalčevih binarnih komponent v spodnjem delu sklada.

Po našem modelu bi LineageOS ocenili kot rešitev z mešanim do pretežno odprtim profilom na ravni sistemskih storitev, privzetih aplikacij in načina

[2] `https://grapheneos.org/`
[3] `https://e.foundation/e-os/`
[4] `https://replicant.us/`

distribucije aplikacij, bistveno slabše pa na ravni strojne programske opreme ter dela gonilnikov. Na ravni praktične uporabnosti je njegov profil močan: osnovne funkcije telefona, aplikacijski ekosistem, splošna odzivnost sistema in možnosti prilagajanja so pri uradno podprtih napravah dovolj dobri za vsakodnevno rabo. V primerjavi s tovarniškimi sistemi LineageOS uporabniku omogoča več nadzora nad nameščeno programsko opremo, več možnosti sistemskih prilagoditev in manjšo odvisnost od proizvajalčevih privzetih storitev. Glavne omejitve se pokažejo pri funkcijah, ki so posebej odvisne od proizvajalčeve implementacije, ter pri aplikacijah, ki preverjajo celovitost naprave ali se opirajo na dodatne zaprtokodne storitve [3].

LineageOS je zato najprimernejši predstavnik "uravnoteženega kompromisa": ne približa se najbolj idealu popolnoma odprtokodnega telefona, je pa ena najbolj uporabnih odprtokodnih alternativ v praksi.

### 5.1.2 GrapheneOS

Projekt GrapheneOS predstavlja drugačno smer razvoja kot LineageOS. Ne osredotoča se na široko podporo številnim napravam, temveč na zasebnost, varnost in tehnično strogost pri ožjem naboru uradno podprtih telefonov Pixel [28, 27]. Ta omejenost ni naključna, temveč je del osnovne zasnove projekta.

GrapheneOS je pomemben predvsem zato, ker pokaže, da večja varnost in večji nadzor nad sistemom ne pomenita nujno tudi večje odprtosti vseh slojev. Na ravni zagonske verige je projekt zelo močan: postopek namestitve je dobro dokumentiran, vezan na uradno podprte naprave, po namestitvi pa predvideva ponovno zaklepanje zagonskega nalagalnika, tako da se ohrani preverjen zagon in zaščita pred nalaganjem nepooblaščenih sprememb sistema [26]. To je pomembna razlika v primerjavi s številnimi drugimi alternativnimi ROM-i, pri katerih uporabnik po namestitvi pogosto ostane pri trajno odklenjenem zagonskem nalagalniku.

Na drugi strani GrapheneOS ni bistveno bolj odprt v spodnjih slojih strojne podpore. Še vedno je vezan na Googlovo strojno osnovo telefonov Pi-

xel, na dobaviteljske komponente strojne programske opreme in na gonilnike, ki jih zagotavlja proizvajalec naprave [27]. Njegova prednost zato ni v tem, da bi odpravil vse zaprtokodne dele, temveč v tem, da na kakovostni in varnostno močni strojni podlagi zgradi strožje nastavljen odprtokoden sistemski sloj.

V dimenziji praktične uporabnosti je GrapheneOS na uradno podprtih napravah zelo močan, saj ohranja združljivost z aplikacijami za sistem Android in hkrati ne zahteva običajnega odpovedovanja ključnim funkcijam telefona. Pomemben del njegove praktične vrednosti je tudi visoka stopnja nadzora nad aplikacijami, dovoljenji in profili uporabe. Po drugi strani pa projekt ni usmerjen v maksimalno prilagodljivost sistema, saj daje prednost ohranjanju strogega varnostnega modela pred poljubnim poseganjem v sistem. Z vidika sklopa uporabniškega nadzora gre zato za rešitev, ki je močna pri nadzoru, vendar bolj omejena pri prostem poseganju v sistem kot nekatere druge odprtokodne alternative. Njegova glavna omejitev je drugje: uporabnik mora sprejeti zelo ozek nabor podprtih naprav in s tem precej večjo vezanost na konkretno strojno platformo. GrapheneOS je zato dober primer rešitve, ki ni najbolj odprta v absolutnem smislu, vendar zelo premišljeno združuje odprtost zgornjih slojev, močan varnostni model in visoko praktično uporabnost.

### 5.1.3 Projekt /e/OS

Projekt /e/OS se od projektov LineageOS in GrapheneOS razlikuje predvsem po tem, da skuša odprtokodno mobilno rešitev približati širšemu krogu uporabnikov, ki jih zanimata zasebnost in manjša odvisnost od Googlovih storitev, ne želijo pa nujno tehnično zahtevnega sistema ali zelo ozkega nabora podprtih naprav [19, 18].

Njegova glavna ideja je različica sistema Android brez Googlovih storitev: odprtokoden sistemski temelj, nadomeščanje Googlovih storitev z alternativami ter bolj uporabniško usmerjen nabor privzetih aplikacij in spremljevalnih storitev. Dokumentacija za podprte naprave pri tem izrecno loči med uradnimi graditvami (angl. *Official*) in skupnostnimi graditvami (angl. *Com-*

*munity*); uradne graditve so po testiranju označene kot primerne za vsakodnevno rabo, obe vrsti pa prejemata sprotne posodobitve sistema po zraku (angl. *over-the-air*, OTA) [19]. To kaže, da projekt jasno cilja na vsakdanjo uporabnost in ne zgolj na tehnično demonstracijo odprtosti.

Po našem modelu je /e/OS pomemben kot rešitev, ki skuša izboljšati odprtost in zasebnost na ravni sistemskih storitev, privzetih aplikacij in uporabniške izkušnje, ne da bi pri tem prekinila združljivost z običajno rabo sistema Android. Tak pristop poveča praktično uporabnost, vendar ne odpravi enakih omejitev v spodnjih slojih kot pri drugih ROM-ih za sistem Android: tudi tu podpora napravam ostaja odvisna od dobaviteljskih komponent, strojne programske opreme in konkretne strojne osnove posameznega telefona.

Zaradi zelo širokega nabora podprtih naprav je profil sistema /e/OS manj enoten kot pri projektu GrapheneOS. To pomeni, da se lahko dejanska kakovost podpore, hitrost posodobitev in obseg kompromisov med napravami precej razlikujejo. Vseeno pa je /e/OS pomemben primer zato, ker pokaže, da je mogoče odprtokodno mobilno rešitev usmeriti predvsem v praktično rabo in ne predvsem v čim večjo tehnično prilagodljivost sistema ali v čim strožje izogibanje lastniškim komponentam. V primerjavi z LineageOS daje /e/OS manjši poudarek globlji prilagodljivosti sistema in izraziteje daje prednost uporabniški dostopnosti. Prav zato je z vidika uporabniškega nadzora manj izrazit predstavnik odprtokodne mobilne rešitve kot projekti, ki so bolj neposredno usmerjeni v tehnično prilagajanje sistema.

### 5.1.4 Replicant

Projekt Replicant je v tej skupini poseben primer, ker kot osnovni cilj postavlja programsko svobodo in zato ne pristaja na številne pragmatične kompromise, ki jih sprejemajo drugi ROM-i za sistem Android. Predstavlja se kot popolnoma prosta distribucija sistema Android, ki ne vključuje in ne priporoča neproste programske opreme [59].

Z vidika idealnega modela iz prejšnjega poglavja je Replicant med ROM-i

za sistem Android najbližje strožji interpretaciji odprtosti. Ravno zato pa zelo jasno pokaže, kako težko je v mobilnem svetu hkrati doseči visoko odprtost in visoko uporabnost. Ker zavračanje lastniških komponent močno omeji nabor uporabne strojne opreme, je seznam podprtih naprav zelo kratek in vezan predvsem na starejše modele [60]. To neposredno vpliva tudi na praktično uporabnost: že sama dostopnost primerne naprave je danes omejitev, poleg tega pa starejša strojna oprema težje zadosti sodobnim pričakovanjem glede zmogljivosti, avtonomije, kamer, omrežne podpore in aplikacijskega ekosistema.

Replicant je zato pomemben predvsem kot referenčni robni primer. Pokaže, da je v okviru sistema Android mogoče zgraditi rešitev, ki programsko svobodo razume bistveno strožje kot projekti LineageOS, GrapheneOS ali /e/OS. Hkrati pa ista odločitev povzroči tako velike omejitve pri podpori napravam in pri vsakdanji rabi, da Replicant za večino uporabnikov danes težko predstavlja realno zamenjavo za sodoben telefon. Po drugi strani je za tehnično bolj podkovanega uporabnika pomemben tudi zato, ker zaradi strožjega odnosa do neproste programske opreme omogoča večji vpogled v sistem in več neposrednega nadzora nad njegovimi odprtimi deli kot bolj pragmatične alternative. Replicant ima lahko zato kljub nizki splošni praktični uporabnosti visoko oceno pri uporabniškem nadzoru, ker prav na tem področju najbolj jasno uresničuje idejo, da ima lastnik naprave dejanski nadzor nad odprtimi deli sistema. Prav zato je za to diplomsko nalogo zelo uporaben: ne kot najbolj praktična rešitev, temveč kot dokaz, kako hitro se ob strožjem razumevanju odprtosti skrči nabor dejansko uporabnih naprav.

### 5.1.5 Vmesna primerjalna ugotovitev

Analiza alternativnih ROM-ov za sistem Android pokaže, da znotraj iste tehnične družine ne obstaja ena sama "najboljša" rešitev, temveč več različnih strategij. LineageOS predstavlja najbolj uravnotežen kompromis med odprtostjo, uporabnostjo in možnostmi prilagajanja sistema. GrapheneOS kaže, kako daleč je mogoče priti z varnostno strogim pristopom, če projekt sprejme

zelo ozek nabor strojne podpore in prilagodljivost sistema podredi strožjemu varnostnemu modelu. Projekt /e/OS daje največji poudarek vsakdanji uporabnosti in zmanjšanju odvisnosti od Googlovega ekosistema, manj pa globljemu uporabniškemu nadzoru nad sistemom. Replicant pa predstavlja najdoslednejšo interpretacijo programske svobode, vendar za ceno zelo omejene praktične uporabnosti.

Skupna omejitev vseh teh rešitev je, da ne odpravijo temeljne odvisnosti od zaprtih komponent v spodnjih slojih mobilne naprave. Njihova največja prednost je zato predvsem v odprtosti uporabniškega prostora, večjem nadzoru nad sistemom, daljši življenjski dobi podprtih naprav in manjši vezanosti na proizvajalčev tovarniški sistem. To je pomemben korak naprej, vendar še ne pomeni, da tak telefon v celoti doseže ideal popolnoma odprtokodnega telefona, kot smo ga opredelili v prejšnjem poglavju.

## 5.2 Mobilne distribucije sistema GNU/Linux na obstoječi strojni opremi za sistem Android

Drugo pomembno skupino odprtokodnih mobilnih rešitev predstavljajo mobilne distribucije sistema GNU/Linux, namenjene delovanju na strojni opremi, ki je bila prvotno razvita za sistem Android. V primerjavi z alternativnimi ROM-i za sistem Android gre tu za drugačen pristop: namesto da bi uporabnik obdržal uporabniški prostor sistema Android in zamenjal predvsem sistemski sloj, skušajo ti projekti na obstoječo mobilno strojno opremo prenesti uporabniški prostor sistema GNU/Linux.

Tak pristop lahko poveča odprtost uporabniškega prostora, preglednost sistema in sorodnost z običajnimi distribucijami GNU/Linux. Hkrati pa ne odpravi samodejno omejitev v spodnjih slojih naprave. Ker je bila večina teh telefonov prvotno razvita za sistem Android, so mobilne distribucije sistema GNU/Linux pogosto še vedno odvisne od spodnjih slojev sistema Android,

prilagojenih jeder, binarnih dodatkov in različnih združljivostnih slojev. Prav zato ta skupina rešitev dobro pokaže, da odprtost uporabniškega prostora še ni enaka odprtosti celotnega sistema.

V tej skupini obravnavamo rešitve Ubuntu Touch[5], Mobian[6], postmarketOS[7] in projekt Halium[8]. Izbrani so zato, ker predstavljajo štiri tesno povezane, vendar vsebinsko različne pristope: Ubuntu Touch kot uporabniško usmerjen mobilni sistem, Mobian kot mobilno distribucijo sistema GNU/Linux, tesno povezano z Debianom, postmarketOS kot širše zasnovano in bolj eksperimentalno mobilno distribucijo sistema GNU/Linux, Halium pa kot združljivostni temelj, ki skuša povezati svet naprav za sistem Android s projekti sistema GNU/Linux.

### 5.2.1 Ubuntu Touch

Projekt Ubuntu Touch je danes eden najpomembnejših predstavnikov mobilnih distribucij sistema GNU/Linux, ki skušajo odprtokodne mobilne rešitve približati tudi manj tehnično usmerjenim uporabnikom. Uradna dokumentacija projekta UBports ga opisuje kot mobilni operacijski sistem, osredotočen na enostavno uporabo, zasebnost in konvergenco [71, 70]. Ta izhodišča kažejo, da projekt ne stavi predvsem na ideološko čistost ali eksperimentalnost, temveč na čim bolj zaokroženo uporabniško izkušnjo.

Ubuntu Touch je pomemben predvsem zato, ker poskuša na mobilni napravi ponuditi uporabniški prostor sistema GNU/Linux, ne da bi se pri tem povsem odpovedal praktični uporabnosti. Na ravni uporabniške izkušnje je projekt bližje klasičnemu razumevanju mobilnega operacijskega sistema kot številni bolj eksperimentalni projekti. Velik poudarek daje konsistentnemu uporabniškemu vmesniku in zaokroženi vsakdanji rabi. Pomemben poudarek je tudi ideja konvergence, torej približevanje mobilnega in namiznega načina uporabe v okviru istega sistema [70].

[5] `https://ubuntu-touch.io/`
[6] `https://mobian.org/`
[7] `https://postmarketos.org/`
[8] `https://halium.org/`

Vendar Ubuntu Touch hkrati zelo jasno pokaže omejitve te skupine rešitev. Dokumentacija za prenašanje sistema na nove naprave izrecno navaja, da je Ubuntu Touch mogoče prenesti na naprave, ki so bile prvotno dobavljene s sistemom Android, pri čemer je večina takih naprav še vedno vsaj delno odvisna od lastniške programske opreme [72]. To pomeni, da Ubuntu Touch bistveno spremeni uporabniški prostor, ne odpravi pa nujno zaprtosti spodnjih slojev strojne podpore.

Po našem modelu bi Ubuntu Touch ocenili kot rešitev, ki je močna na ravni odprtosti uporabniškega prostora, preglednosti sistema in nadzora nad privzetimi storitvami, šibkejša pa na ravni gonilnikov, strojne programske opreme in celotne strojne neodvisnosti. Na ravni prilagodljivosti in uporabniškega nadzora ponuja več možnosti kot tipični potrošniški mobilni sistemi, vendar zaradi manj zrelega ekosistema in projektno specifičnih rešitev ne dosega enake neposrednosti kot klasične mobilne distribucije GNU/Linux. V primerjavi z alternativnimi ROM-i za sistem Android je njegov položaj obraten: v nekaterih vidikih je bližje idealu odprtega sistema, vendar težje dosega enako raven aplikacijske združljivosti in splošne vsakdanje uporabnosti.

### 5.2.2 Mobian

Projekt Mobian predstavlja nekoliko drugačen pristop kot Ubuntu Touch, čeprav sodi v isto širšo skupino mobilnih distribucij sistema GNU/Linux. Na uradni strani se predstavlja kot izpeljanka Debiana za mobilne naprave, uradni blog projekta pa poudarja, da želi standardno distribucijo Debian povezati z mobilnimi prilagoditvami, pri čemer skuša čim več posebnih rešitev prenesti v izvorne projekte in s tem zmanjšati količino lastnih, projektno specifičnih sprememb [44, 45]. Že iz tega izhodišča je razvidno, da Mobian ne želi biti povsem ločen mobilni ekosistem, temveč čim bolj neposredna mobilna razširitev obstoječega ekosistema Debian.

Mobian je pomemben predvsem zato, ker je med obravnavanimi rešitvami eden najbližjih predstavnikov mobilnega sistema, ki ostaja tesno povezan z

običajno distribucijo GNU/Linux. Tak pristop lahko poveča preglednost sistema, zmanjša količino projektno specifičnih prilagoditev in olajša razumevanje celotnega programskega sklada. Hkrati pa pomeni, da projekt manj stavi na strogo nadzorovano in poenoteno mobilno uporabniško izkušnjo kot Ubuntu Touch, temveč bolj na postopno prilagajanje obstoječega ekosistema GNU/Linux mobilni rabi.

Za to diplomsko nalogo je Mobian pomemben tudi zato, ker ni omejen le na namenske naprave z GNU/Linuxom, temveč podpira tudi nekatere telefone, prvotno razvite za sistem Android. Debianova dokumentacija je leta 2025 med uradno podprtimi napravami, ki so bile prvotno razvite za sistem Android, navajala OnePlus 6/6T in Pocophone F1 [17]. S tem Mobian jasno sodi v skupino rešitev, ki skušajo na obstoječo strojno opremo, prvotno razvito za sistem Android, prenesti uporabniški prostor sistema GNU/Linux.

Po našem modelu bi Mobian ocenili kot rešitev, ki je močna pri odprtosti uporabniškega prostora, bližini običajnim distribucijam GNU/Linux, preglednosti programskega sklada ter možnostih prilagajanja in uporabniškega nadzora. Na ravni splošne praktične uporabnosti pa je njegov profil manj enoten. V primerjavi z Ubuntu Touch je manj osredotočen na čim bolj zaokroženo mobilno uporabniško izkušnjo, v primerjavi s postmarketOS pa deluje bolj konservativno in bližje klasični logiki distribucije Debian. Prav zato je Mobian dober vmesni primer med uporabniško usmerjenim mobilnim sistemom in bolj eksperimentalnim prilagajanjem sistema GNU/Linux širšemu naboru naprav.

### 5.2.3 postmarketOS

Projekt postmarketOS predstavlja bolj izrazito eksperimentalno interpretacijo mobilnega sistema GNU/Linux kot Ubuntu Touch. Njegov cilj ni predvsem zgraditi čim bolj podoben nadomestek za običajen pametni telefon, temveč mobilno strojno opremo približati običajni logiki distribucij GNU/Linux in dolgoročnega vzdrževanja. Že uradna predstavitev projekta poudarja, da gre za razširitev distribucije Alpine Linux na pametne tele-

fone [51].

Projekt je posebej zanimiv zato, ker zelo jasno poudari razliko med široko podporo napravam in dejansko stopnjo uporabnosti. Seznam podprtih naprav je izredno velik; uradna dokumentacija je marca 2026 navajala 735 podprtih naprav [53]. Vendar sama podprtost še ne pomeni, da posamezna naprava ponuja enako stopnjo praktične uporabnosti. Pri številnih napravah to pomeni predvsem osnovno stopnjo podpore, medtem ko so posamezne funkcije, kot so mobilna omrežja, kamera, pospeševanje grafike ali upravljanje porabe, še vedno omejene.

Projekt postmarketOS je zato pomemben kot primer rešitve, ki močno razširi prostor eksperimentiranja in dolgoročnega oživljanja starejših naprav, vendar prav zaradi te širine težje zagotovi enotno uporabniško izkušnjo. Na ravni odprtosti uporabniškega prostora je zelo močan, na ravni praktične uporabnosti pa je profil med napravami bistveno bolj neenoten kot pri Ubuntu Touch ali pri alternativnih ROM-ih za sistem Android. Za tehnično bolj podkovanega uporabnika je postmarketOS posebej zanimiv tudi zato, ker omogoča visoko stopnjo prilagodljivosti in neposrednega nadzora nad sistemom. Vendar je praktična vrednost takšne rešitve močno odvisna od tega, kako zrela je prilagoditev za posamezno napravo in koliko ključnih funkcij je na izbrani strojni opremi dejansko uporabnih.

### 5.2.4 Vloga projekta Halium in sorodnih združljivostnih slojev

Projekt Halium sam po sebi ni končni mobilni operacijski sistem za vsakdanjo rabo, temveč združljivostni temelj za projekte, ki skušajo na naprave s prednameščenim sistemom Android prenesti uporabniški prostor sistema GNU/Linux. Uradna dokumentacija ga opredeli kot sodelovalni projekt za poenotenje abstrakcijske plasti strojne opreme za projekte, ki na mobilnih napravah s prednameščenim sistemom Android poganjajo GNU/Linux [29].

Za to diplomsko nalogo je Halium pomemben predvsem kot razlaga, zakaj mobilne distribucije sistema GNU/Linux na obstoječi strojni opremi pogo-

sto ne morejo preprosto prekiniti vezi z okoljem sistema Android. Namesto popolne zamenjave spodnjih slojev sistema pogosto uporabljajo vmesne združljivostne rešitve, ki omogočijo uporabo obstoječih Androidovih komponent v drugačnem uporabniškem prostoru [29, 36, 62]. To pomeni, da je lahko uporabniški del sistema bistveno bolj odprt in bolj podoben običajnim distribucijam GNU/Linux, medtem ko spodnji del sklada ostaja tehnično in licenčno precej bolj hibriden.

### 5.2.5 Vmesna primerjalna ugotovitev

Analiza mobilnih distribucij sistema GNU/Linux na obstoječi strojni opremi za sistem Android pokaže drugačen profil kompromisov kot pri alternativnih ROM-ih za sistem Android. Medtem ko so alternativni ROM-i navadno močnejši pri vsakdanji uporabnosti in združljivosti z mobilnim ekosistemom, so mobilne distribucije sistema GNU/Linux zanimive predvsem z vidika odprtosti uporabniškega prostora, preglednosti sistema, prilagodljivosti in dolgoročnega raziskovalnega potenciala.

Kljub skupnemu izhodišču pa se znotraj te skupine pokažejo pomembne razlike med posameznimi projekti. Ubuntu Touch daje največji poudarek zaokroženi mobilni uporabniški izkušnji, Mobian ostaja najtesneje povezan z običajno distribucijo Debian, postmarketOS pa najbolj odpira prostor za prilagajanje sistema različnim napravam, eksperimentiranje in neposredni uporabniški nadzor. Ob tem je za celotno skupino pomembno, da pri delovanju na obstoječi strojni opremi pogosto še vedno ostaja vsaj delno odvisna od spodnjih slojev, ki izhajajo iz okolja sistema Android. To omejitev posebej jasno osvetli vloga projekta Halium.

Prav ta skupina zato najbolj nazorno pokaže, da je mobilna strojna oprema še vedno močno zaznamovana z dediščino sistema Android. Prehod na uporabniški prostor sistema GNU/Linux sam po sebi še ne pomeni popolnoma odprte naprave, pogosto pa vseeno poveča stopnjo dejanskega nadzora uporabnika nad sistemom.

## 5.3 Namenski telefoni z operacijskim sistemom GNU/Linux

Tretjo skupino obravnavanih rešitev predstavljajo namenski telefoni z operacijskim sistemom GNU/Linux. V nasprotju z alternativnimi ROM-i za sistem Android in mobilnimi distribucijami sistema GNU/Linux na obstoječi strojni opremi ne gre več za naknadno prilagajanje ali zamenjavo programske opreme na napravah, ki so bile prvotno razvite za drug ekosistem. Pri tej skupini sta tako strojna kot programska zasnova že od začetka usmerjeni v čim večji nadzor uporabnika, tesnejšo povezanost z ekosistemom GNU/Linux in manjšo odvisnost od običajnega mobilnega programskega sklada, kakršen je uveljavljen pri napravah s sistemom Android.

Tak pristop je z vidika modela iz prejšnjega poglavja posebej zanimiv zato, ker se lahko najbolj približa idealu popolnoma odprtokodnega telefona. Ker naprava ni zasnovana kot kompromisna prilagoditev obstoječemu telefonu s sistemom Android, je mogoče doseči večjo preglednost arhitekture, bolj neposreden nadzor nad posameznimi podsistemi in v nekaterih primerih tudi boljšo fizično ločitev komponent, kot so modem, brezžične povezave in senzorji. Po drugi strani pa takšne naprave težje konkurirajo običajnim pametnim telefonom glede zmogljivosti, energijske učinkovitosti, kakovosti kamer, zrelosti aplikacijskega ekosistema in splošne vsakdanje uporabnosti.

V tej skupini kot osrednja primera obravnavamo PinePhone[9] in Librem 5[10]. Izbrana sta zato, ker sodita med najbolj uveljavljene in najizrazitejše predstavnike namenskih telefonov z operacijskim sistemom GNU/Linux. Na koncu na kratko omenimo še nekatere novejše oziroma bolj robne projekte, kot so Jolla C2, FuriPhone FLX1s in Liberux NEXX, ki kažejo, da to področje ostaja razvojno aktivno, vendar za zdaj še ne predstavljajo tako uveljavljene referenčne osnove kot PinePhone in Librem 5.

[9] `https://pine64.org/devices/pinephone/`
[10] `https://puri.sm/products/librem-5/`

### 5.3.1 PinePhone

Projekt PinePhone predstavlja enega najizrazitejših primerov telefona, ki je bil od začetka zamišljen kot naprava za skupnost mobilnih projektov GNU/Linux. Uradna predstavitev ga opisuje kot telefon, ki zmore poganjati glavno vejo jedra Linux, vključuje strojna stikala za zasebnost in ga podpirajo glavni projekti mobilnega GNU/Linux [49, 50]. Že ta opis pokaže, da se PinePhone ne poskuša približati običajni logiki komercialnega pametnega telefona, temveč predvsem ponuditi odprto in eksperimentalno razvojno platformo.

Ključna prednost PinePhona je, da naprava ni zasnovana okoli sistema Android, temveč okoli neposredne podpore za GNU/Linux. Podjetje PINE64, ki se predstavlja kot skupnostno usmerjeno podjetje, pri tem posebej poudarja podporo glavni veji jedra Linux in prisotnost strojnih stikal za zasebnost [49]. To pomeni, da je nadzor nad napravo v večji meri prenesen na uporabnika in skupnost razvijalcev, ne pa na proizvajalca zaprtega mobilnega ekosistema.

Vendar PinePhone hkrati zelo jasno pokaže, kako težko je visoko stopnjo odprtosti pretvoriti v vsakdanje uporabno napravo. Njegova zasnova je izrecno naklonjena eksperimentiranju, odprtemu razvoju in podpori različnih mobilnih distribucij sistema GNU/Linux, vendar to ne pomeni, da vse ključne funkcije dosegajo enako raven zrelosti kot pri običajnih pametnih telefonih. Prav zato je PinePhone pomemben predvsem kot referenčna razvojna naprava in kot dokaz, da je mogoče zgraditi telefon, ki ni podrejen logiki sistema Android ali iOS. Kot splošna rešitev za vsakdanjo rabo pa ostaja manj prepričljiv.

Po našem modelu bi PinePhone ocenili kot rešitev, ki je zelo močna na ravni arhitekturne odprtosti, preglednosti sistema, prilagodljivosti in nadzora nad strojno opremo, šibkejša pa na ravni praktične uporabnosti. Njegova največja prednost je prav to, da gre za namensko napravo za mobilni GNU/Linux, njegova največja slabost pa je, da ta prednost še ne pomeni tudi enake stopnje zrelosti kot pri uveljavljenih komercialnih telefonih.

### 5.3.2 Librem 5

Librem 5 predstavlja drugačno interpretacijo namenskega telefona z operacijskim sistemom GNU/Linux kot PinePhone. Če PinePhone bolj odkrito cilja na razvojno skupnost in eksperimentiranje, Purism pri Librem 5 veliko izraziteje poudarja zasebnost, varnost in nadzor uporabnika. Uradna stran naprave ga opisuje kot telefon, zgrajen na sistemu PureOS, ki ni osnovan na sistemu Android ali iOS, dokumentacija in varnostna predstavitev pa posebej poudarjata tri strojna stikala za fizični izklop komponent (angl. *kill switches*) ter tako imenovani način popolne omejitve *Lockdown mode* [55, 57, 56].

Librem 5 je posebej zanimiv zato, ker odprtost povezuje z bolj premišljeno varnostno zasnovo. Tri strojna stikala omogočajo fizični izklop mobilnega modema, Wi-Fi/Bluetooth modula ter kamere in mikrofona, *Lockdown mode* pa ob hkratni uporabi vseh treh stikal izklopi še druge senzorje, kot so GPS, pospeškomer, senzor bližine in svetlobni senzor [57, 56]. Takšna rešitev je v mobilnem prostoru zelo neobičajna, saj uporabniku daje neposreden fizični nadzor nad komponentami, ki so pri običajnih pametnih telefonih večinoma skrite za programskimi plastmi.

V primerjavi s PinePhonom je Librem 5 manj zanimiv kot povsem odprta razvojna platforma v ožjem smislu, bolj pa kot naprava, ki skuša odprtost in svobodo povezati z uporabniško razumljivimi mehanizmi nadzora. Njegova glavna prednost ni zgolj v tem, da poganja GNU/Linux, temveč v tem, da svojo zasnovo gradi okoli ideje, da mora imeti uporabnik nadzor tudi na ravni fizične strojne opreme. To ga približa idealu popolnoma odprtokodnega telefona z druge strani kot PinePhone: manj skozi eksperimentalno odprtost, bolj skozi zasebnostno in varnostno usmerjeno arhitekturo.

Po našem modelu bi Librem 5 ocenili kot rešitev, ki je zelo močna na ravni nadzora uporabnika, preglednosti arhitekture, ločevanja posameznih podsistemov ter prilagodljivosti sistema, manj prepričljiva pa ostaja na ravni vsakdanje uporabnosti in konkurenčnosti v primerjavi z običajnimi sodobnimi pametnimi telefoni. Prav v tem je tudi njegova glavna vrednost za to diplomsko nalogo: pokaže, da je mogoče zasnovati telefon, ki ni le programsko

drugačen, temveč tudi strojno organiziran v korist večje avtonomije uporabnika.

### 5.3.3 Drugi novejši oziroma robni projekti

Poleg PinePhona in Librem 5 je v zadnjih letih mogoče opaziti še nekaj novejših ali bolj robnih projektov, ki poskušajo nadaljevati idejo telefona z operacijskim sistemom GNU/Linux. Med njimi velja omeniti Jolla C2[11], FuriPhone FLX1s[12] in Liberux NEXX[13].

Jolla C2 je zanimiv predvsem kot referenčna naprava za skupnost sistema Sailfish OS. Uradna predstavitev ga opisuje kot referenčno napravo za skupnost sistema Sailfish OS z 4G in VoLTE podporo ter možnostjo poganjanja aplikacij za Android [31, 32]. Vendar je za to diplomsko nalogo pomembna tudi omejitev, da uradna dokumentacija Sailfish OS sama navaja, da sistem sestavljajo tako odprtokodne kot tudi zaprte komponente [63]. Jolla C2 je zato zanimiv kot pokazatelj, da na trgu še vedno obstaja komercialni interes za telefone z GNU/Linuxom, ni pa tako "čist" primer odprtokodne mobilne rešitve kot PinePhone ali Librem 5.

FuriPhone FLX1s predstavlja novejši poskus telefona, ki temelji na sistemu FuriOS, opisanem kot mobilno prilagojen sistem na osnovi Debiana. Proizvajalec pri njem poudarja tri strojna stikala za zasebnost in usmerjenost v mobilno uporabo brez umetnih omejitev [21, 22]. Ta projekt je zanimiv zato, ker skuša združiti ideje klasičnega okolja GNU/Linux, zasebnosti in vsakdanje uporabnosti, vendar je za zdaj še premlad, da bi ga lahko obravnavali kot enako referenčen primer kot PinePhone ali Librem 5.

Liberux NEXX pa predstavlja še zgodnejši primer novega projekta v nastajanju. Uradna stran ga opisuje kot telefon, zgrajen na sistemu LiberuxOS, ki je opisan kot etičen, prost in večinoma odprtokoden ter ni osnovan na Androidu ali iOS, spremljevalne objave projekta pa govorijo o množičnem

[11] `https://commerce.jolla.com/products/jolla-community-phone`
[12] `https://furilabs.com/shop/flx1s/`
[13] `https://liberux.net/`

financiranju in prototipiranju prve razvojne plošče [33, 34, 35]. Takšni projekti so pomembni predvsem zato, ker kažejo, da ideja namenskega telefona z GNU/Linuxom še ni izginila, vendar je njihova dejanska dolgoročna uspešnost v tej fazi še negotova.

### 5.3.4 Vmesna primerjalna ugotovitev

Analiza namenskih telefonov z operacijskim sistemom GNU/Linux pokaže, da se ta skupina najbolj približa idealu popolnoma odprtokodnega telefona, vendar za ceno nižje stopnje praktične uporabnosti. PinePhone predstavlja izrazitejši razvojno-skupnostni pol, pri katerem so v ospredju odprta arhitektura, visoka prilagodljivost sistema in podpora različnim projektom mobilnega GNU/Linux. Librem 5 pa predstavlja varnostno in zasebnostno bolj usmerjen pol, pri katerem so v ospredju fizični mehanizmi nadzora nad strojno opremo in jasneje izražena ideja uporabniške avtonomije.

Novejši projekti, kot so Jolla C2, FuriPhone FLX1s in Liberux NEXX, kažejo, da področje namenskih telefonov z GNU/Linuxom ostaja razvojno živo, vendar so ti primeri bodisi manj dosledni z vidika odprtokodnosti bodisi še premalo zreli, da bi že predstavljali enako stabilno referenčno osnovo kot PinePhone in Librem 5.

Ta skupina rešitev zato najbolje pokaže osrednjo napetost celotne diplomske naloge. Bolj ko se mobilna rešitev približa idealu odprte arhitekture, neposrednega nadzora uporabnika in prilagodljivega sistema, težje dosega enako raven zrelosti, udobja in splošne vsakdanje uporabnosti kot prevladujoči komercialni pametni telefoni.

## 5.4 Primerjalna sinteza

Analiza treh skupin odprtokodnih mobilnih rešitev pokaže, da se nobena izmed obravnavanih rešitev ne približa idealu popolnoma odprtokodnega telefona hkrati v obeh dimenzijah, ki smo ju opredelili v prejšnjem poglavju. Pri vseh treh skupinah se pokaže izrazita napetost med odprtostjo in praktično

uporabnostjo. Rešitve, ki uporabniku omogočajo večji nadzor nad arhitekturo sistema, večjo prilagodljivost in bolj odprto programsko okolje, težje dosežejo enako raven vsakdanje uporabnosti kot rešitve, ki ostajajo bliže prevladujočemu ekosistemu sistema Android.

Sintezne ocene odprtosti in praktične uporabnosti so povzete v tabelah 5.1 in 5.2, dodatno pa jih ponazarjata radarska diagrama na slikah 5.1 in 5.2. Ocene v tabelah so sintezne in temeljijo na analizi javno dostopne dokumentacije, znanih omejitev in razvojnih usmeritev posameznih projektov. Gre torej za okvirno primerjavo profilov posameznih rešitev, ne za strogo merilno vrednotenje.

| **Rešitev** | **ZV** | **JG** | **SP** | **SS** | **AD** |
|---|---|---|---|---|---|
| LineageOS | 1 | 1 | 0 | 1 | 1 |
| GrapheneOS | 2 | 1 | 0 | 1 | 1 |
| /e/OS | 1 | 1 | 0 | 1 | 1 |
| Replicant | 1 | 1 | 1 | 2 | 1 |
| Ubuntu Touch | 1 | 1 | 0 | 1 | 2 |
| Mobian | 1 | 1 | 0 | 2 | 2 |
| postmarketOS | 1 | 1 | 0 | 2 | 2 |
| PinePhone | 2 | 2 | 1 | 2 | 2 |
| Librem 5 | 2 | 2 | 1 | 2 | 2 |

Tabela 5.1: Sintezna primerjava odprtosti po slojih. Oznake: ZV – zagonska veriga, JG – jedro in gonilniki, SP – strojna programska oprema, SS – sistemske storitve in vmesni sloj, AD – privzete aplikacije in distribucija aplikacij. Vrednosti pomenijo: 0 – pretežno zaprto, 1 – mešano, 2 – pretežno odprto.

| **Rešitev** | **OF** | **KS** | **AZ** | **EA** | **PV** | **UN** |
|---|---|---|---|---|---|---|
| LineageOS | 2 | 2 | 2 | 2 | 2 | 2 |
| GrapheneOS | 2 | 2 | 2 | 2 | 2 | 1 |
| /e/OS | 2 | 2 | 2 | 2 | 1 | 1 |
| Replicant | 1 | 0 | 0 | 0 | 0 | 2 |
| Ubuntu Touch | 1 | 1 | 1 | 1 | 2 | 1 |
| Mobian | 1 | 1 | 1 | 1 | 1 | 2 |
| postmarketOS | 1 | 1 | 0 | 1 | 1 | 2 |
| PinePhone | 1 | 1 | 0 | 1 | 1 | 2 |
| Librem 5 | 1 | 1 | 0 | 1 | 2 | 2 |

Tabela 5.2: Sintezna primerjava praktične uporabnosti po funkcionalnih sklopih. Oznake: OF – osnovne funkcije telefona, KS – ključna strojna podpora, AZ – avtonomija in zmogljivost, EA – ekosistem aplikacij, PV – posodobitve in vzdrževanje, UN – uporabniški nadzor. Vrednosti pomenijo: 0 – neuporabno, 1 – omejeno, 2 – uporabno.

Visoka ocena v posameznem sklopu še ne pomeni visoke splošne uporabnosti rešitve, temveč kaže predvsem to, da je rešitev močna v točno določeni dimenziji praktične uporabnosti. Tabeli 5.1 in 5.2 zato ne kažeta ene same lestvice "najboljše" rešitve, temveč različne profile kompromisov.

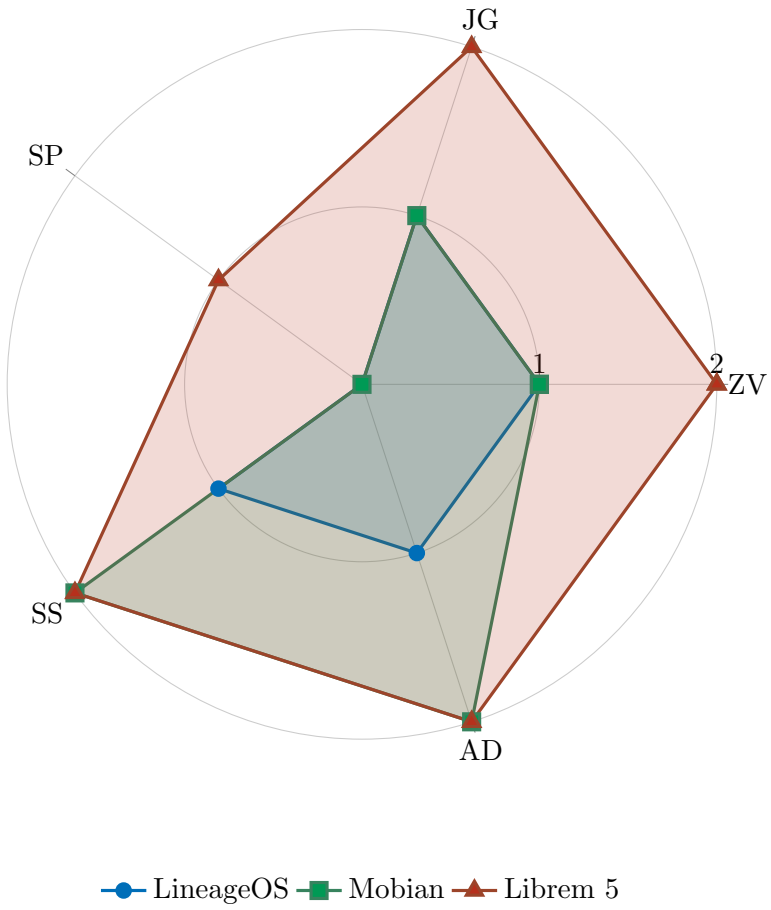
Iz tabel je najprej razvidno, da alternativni ROM-i za sistem Android dosegajo najvišje ocene praktične uporabnosti, vendar ostajajo omejeni v spodnjih slojih sistema, zlasti pri strojni programski opremi in delu gonilnikov. Mobilne distribucije sistema GNU/Linux na obstoječi strojni opremi za sistem Android izboljšajo odprtost uporabniškega prostora in pogosto tudi preglednost sistema, vendar ne morejo v celoti prekiniti odvisnosti od izvorne zasnove naprav za sistem Android. Namenski telefoni z operacijskim sistemom GNU/Linux so v arhitekturnem smislu najbližje idealu odprtega telefona, vendar pri praktični uporabnosti zaostajajo za rešitvami, ki so tesneje povezane z ekosistemom sistema Android.

Takšna primerjava ni pomembna le z vidika odprtosti, temveč tudi z vi-

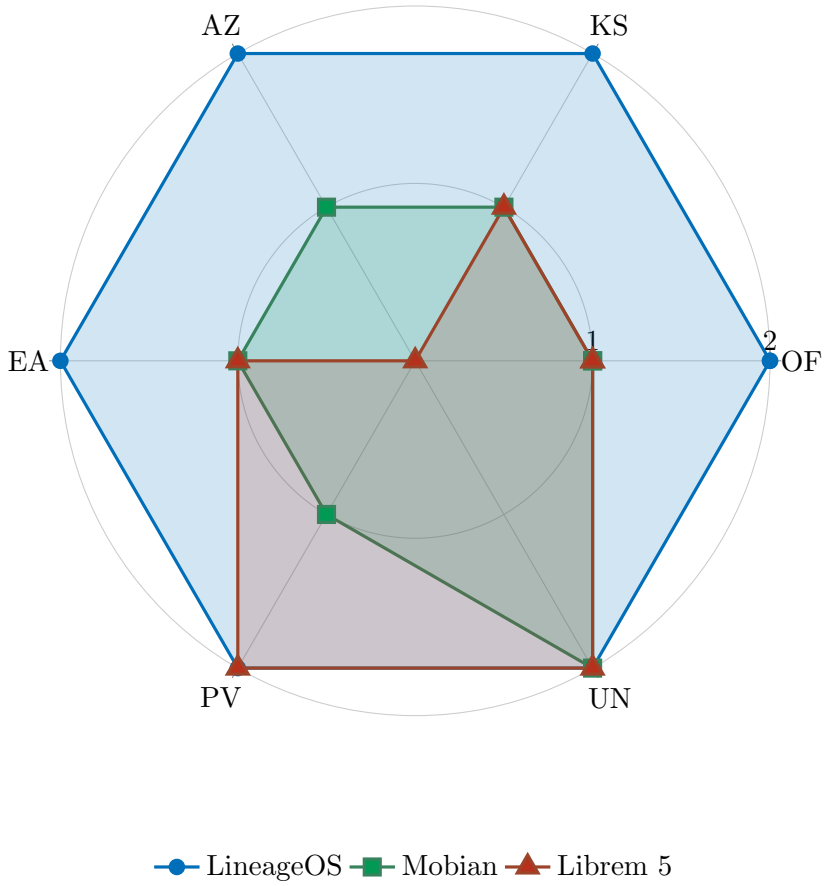
dika zaupanja. Zaprtost spodnjih slojev sistema pomeni, da uporabnik in skupnost ne moreta neodvisno preveriti vseh komponent, ki se izvajajo na napravi. Vprašanje stranskih vrat je zato posebej pomembno pri podsistemih z visokimi privilegiji, kot so modem, brezžični vmesniki in drugi deli dobaviteljskega sklada, kjer se omejitve odprtosti neposredno povezujejo tudi z omejitvami preverljivosti.

Dodani sklop uporabniškega nadzora pomaga razliko med skupinami rešitev pokazati še jasneje. Pri nekaterih projektih je glavni cilj predvsem ponuditi bolj uporabno alternativo tovarniškemu sistemu, pri drugih pa pomemben del vrednosti predstavlja prav večji neposredni nadzor nad sistemom. Najizraziteje se to pokaže pri rešitvah, ki so tesneje povezane z običajnim okoljem GNU/Linux, medtem ko uporabniško dostopnejši Android ROM-i ta vidik pogosteje podredijo širši združljivosti in enostavnejši uporabi.

Zaradi preglednosti radarska diagrama na slikah 5.1 in 5.2 ne prikazujeta vseh obravnavanih rešitev, temveč tri reprezentativne profile. To so LineageOS kot predstavnik alternativnih ROM-ov za sistem Android, Mobian kot predstavnik mobilnih distribucij sistema GNU/Linux na obstoječi strojni opremi za sistem Android ter Librem 5 kot predstavnik namenskih telefonov z operacijskim sistemom GNU/Linux.




Slika 5.1: Radarski diagram odprtosti za tri reprezentativne profile. Oznake osi so enake kot v tabeli 5.1.




Slika 5.2: Radarski diagram praktične uporabnosti za tri reprezentativne profile. Oznake osi so enake kot v tabeli 5.2.

Radarska diagrama potrjujeta ugotovitve iz tabel, vendar jih hkrati pokažeta bolj pregledno kot same številčne ocene. LineageOS se kot predstavnik alternativnih ROM-ov za sistem Android izkaže z najbolj uravnoteženim profilom praktične uporabnosti, vendar ostaja omejen na ravni strojne programske opreme in dela spodnjih slojev sistema. Mobian kot predstavnik mobilnih distribucij sistema GNU/Linux na obstoječi strojni opremi kaže izrazitejšo odprtost uporabniškega prostora ter večjo prilagodljivost in uporabniški nadzor, vendar brez enake stopnje vsakdanje zrelosti kot LineageOS. Librem 5 pa se kot namenska naprava najbolj približa idealu odprte arhitekture, pri praktični uporabnosti pa ostaja v bolj omejenem območju, čeprav uporabniku omogoča zelo neposreden nadzor nad sistemom in strojno opremo.

Če tri skupine rešitev primerjamo z vidika odprtosti, so alternativni ROM-i za sistem Android najmočnejši v zgornjih slojih sistema, torej pri uporabniškem prostoru, sistemskih storitvah in distribuciji aplikacij. Njihova glavna omejitev je, da ne odpravijo temeljne odvisnosti od zaprtih komponent v spodnjih slojih naprave. Mobilne distribucije sistema GNU/Linux na obstoječi strojni opremi za sistem Android povečajo odprtost uporabniškega prostora in tesnejšo povezanost z običajnimi distribucijami GNU/Linux, vendar pogosto ostajajo tehnično odvisne od jeder, binarnih dodatkov in združljivostnih slojev, ki izhajajo iz prvotne zasnove naprav za sistem Android. Namenski telefoni z operacijskim sistemom GNU/Linux se idealu odprtega telefona približajo najbolj neposredno, saj omogočajo bolj pregleden odnos med strojno in programsko zasnovo ter več neposrednega nadzora nad posameznimi podsistemi.

Če iste rešitve primerjamo z vidika praktične uporabnosti, se razmerje med skupinami deloma obrne. Najbolj prepričljivi ostajajo alternativni ROM-i za sistem Android, saj v največji meri ohranijo združljivost z aplikacijskim ekosistemom sistema Android in s tem tudi osnovne prednosti sodobnega pametnega telefona. To pa ne velja brez izjem, zlasti pri aplikacijah, ki uporabljajo mehanizme preverjanja celovitosti naprave [3]. Mobilne distribucije sistema GNU/Linux na obstoječi strojni opremi za sistem Android so manj eno-

tne. Ubuntu Touch daje največji poudarek zaokroženi mobilni uporabniški izkušnji, Mobian ostaja najbližje logiki običajne distribucije GNU/Linux, postmarketOS pa najbolj razširja prostor eksperimentiranja, dolgoročnega oživljanja starejših naprav ter neposrednega uporabniškega nadzora. Namenski telefoni z operacijskim sistemom GNU/Linux so pri praktični uporabnosti najmanj konkurenčni v primerjavi z običajnimi pametnimi telefoni. Njihova glavna vrednost je drugje, in sicer v preizkušanju drugačnega modela naprave, pri katerem ima uporabnik več neposrednega nadzora nad sistemom in strojno opremo.

Primerjalna sinteza tako potrjuje osnovno izhodišče te diplomske naloge. V sodobnem mobilnem prostoru odprtost, prilagodljivost in praktična uporabnost niso lastnosti, ki bi jih bilo mogoče hkrati maksimirati brez pomembnih kompromisov. Bolj ko se rešitev približuje idealu odprte arhitekture in večjega uporabniškega nadzora, več omejitev se pokaže pri zrelosti programske podpore, aplikacijskem ekosistemu in vsakdanji rabi. Po drugi strani pa rešitve, ki so uporabniku najbližje kot dejanska zamenjava za običajen pametni telefon, skoraj vedno ostajajo odvisne od zaprtih komponent v spodnjih slojih sistema.

Takšna primerjalna slika je pomembna tudi za nadaljnji empirični del naloge. Omogoča namreč, da izbrano rešitev v nadaljevanju ne razumemo kot izoliran primer, temveč kot del širšega prostora kompromisov med odprtostjo, varnostjo, vzdrževanjem, prilagodljivostjo in praktično uporabnostjo.

# Poglavje 6

# Empirični del: študija primera na telefonu OnePlus 6T

V prejšnjem poglavju smo odprtokodne mobilne rešitve primerjali predvsem na podlagi javno dostopne dokumentacije, razvojnih usmeritev projektov in znanih omejitev. Takšna primerjava je uporabna za pregled širšega prostora rešitev, ne more pa v celoti nadomestiti neposrednega preizkusa na konkretni napravi. Zato v empiričnem delu izvedemo študijo primera na telefonu OnePlus 6T, na katerem primerjamo štiri sisteme: tovarniški sistem OxygenOS, LineageOS, Ubuntu Touch in postmarketOS.

Namen empiričnega dela ni ugotoviti, kateri projekt je "najboljši" v splošnem smislu, temveč preveriti, kako se izbrane rešitve obnašajo na isti strojni osnovi. Na ta način lahko bistveno bolj neposredno opazujemo kompromise, ki smo jih v prejšnjih poglavjih obravnavali na teoretični ravni. Posebej nas zanima, kako se na konkretni napravi spreminja razmerje med odprtostjo, praktično uporabnostjo, zrelostjo programske podpore in dejanskim nadzorom uporabnika nad sistemom.

V skladu z modelom iz poglavja 4 tudi v empiričnem delu posameznih rešitev ne obravnavamo z eno samo skupno oceno. Sisteme ocenjujemo ločeno v dveh dimenzijah: odprtosti in praktični uporabnosti. Odprtost presojamo po slojih sistema, praktično uporabnost pa po funkcionalnih sklopih, ki smo

jih opredelili v sekcijah 4.3 in 4.4.

## 6.1 Cilj in zasnova študije

Študija primera je zasnovana kot primerjava štirih različnih pristopov na isti napravi. Tovarniški sistem OxygenOS predstavlja izhodiščno referenco, torej stanje, kakršnega je telefon dosegal v okviru proizvajalčeve uradne programske podpore. LineageOS predstavlja pragmatično odprtokodno alternativo znotraj ekosistema sistema Android. Ubuntu Touch in postmarketOS pa predstavljata dva različna pristopa mobilnega sistema GNU/Linux na isti strojni opremi.

Tak izbor je primeren zato, ker na isti napravi zajame tri različne profile kompromisov. OxygenOS je referenca najvišje pričakovane združljivosti s strojno opremo in aplikacijami. LineageOS pokaže, koliko praktične uporabnosti je mogoče ohraniti ob večjem nadzoru nad sistemom in brez proizvajalčeve preobleke. Ubuntu Touch in postmarketOS pa pokažeta, kaj se zgodi, ko se od uporabniškega prostora sistema Android premaknemo proti mobilnemu sistemu GNU/Linux.

## 6.2 Testna naprava

Testna naprava je OnePlus 6T, kodno ime `fajita`. Gre za telefon, predstavljen leta 2018, z vezjem Qualcomm Snapdragon 845, zaslonom Optic AMOLED velikosti 6,41 palca, 6 GB ali 8 GB delovnega pomnilnika, baterijo kapacitete 3700 mAh, modulom NFC, priključkom USB-C in čitalnikom prstnih odtisov pod zaslonom [46].

Izbira te naprave je smiselna iz več razlogov. Prvič, strojna oprema je kljub starosti še vedno dovolj zmogljiva, da razlike med sistemi niso zgolj posledica prešibke naprave. Drugič, OnePlus 6T omogoča odklep zagonskega nalagalnika in je že dalj časa priljubljena naprava v skupnosti alternativnih sistemov. Tretjič, gre za tipičen primer komercialnega telefona z Androidom,

pri katerem je uporabniški prostor mogoče razmeroma enostavno zamenjati, medtem ko spodnji sloji sistema ostajajo močno odvisni od zaprtih komponent.

Za namen diplomskega dela je ta naprava zato zanimiva prav kot realistično izhodišče. Ne gre za namenski telefon z GNU/Linuxom, temveč za komercialno napravo, na kateri želimo preveriti, kako daleč lahko v praksi pridemo z odprtokodnimi alternativami.

## 6.3 Izbrani sistemi

### 6.3.1 Tovarniški sistem OxygenOS

Kot izhodiščni sistem uporabimo tovarniški sistem OxygenOS za OnePlus 6T. Ta sistem predstavlja referenčno točko, saj je bil izrecno prilagojen strojni opremi naprave in je zato pričakovano najmočnejši pri združljivosti s podsistemi, kot so kamera, modem, čitalnik prstnih odtisov, upravljanje porabe energije in aplikacijski ekosistem. Za OnePlus 6 in OnePlus 6T je bila javno objavljena izdaja OxygenOS 11.1.2.2 na osnovi Androida 11 [47].

V empiričnem delu je vloga sistema OxygenOS predvsem primerjalna. Ne obravnavamo ga kot odprtokodno rešitev, temveč kot praktični referenčni maksimum za konkretno strojno opremo. Prav primerjava z njim omogoča, da kasneje bolj jasno ocenimo, katere funkcionalnosti alternativni sistemi ohranijo, katere izgubijo in kje se kompromisi pokažejo najbolj izrazito.

### 6.3.2 LineageOS

LineageOS je v tej študiji predstavnik alternativnih ROM-ov za sistem Android. Za OnePlus 6T obstaja uradna stran naprave v dokumentaciji projekta LineageOS, aktualna navodila za namestitev pa se nanašajo na napravo `fajita` in na izdajo LineageOS 22.2 [42, 40].

Izbira LineageOS je pomembna zato, ker gre za verjetno najbolj realističen odprtokodni kompromis za vsakodnevno rabo na obstoječi napravi

Android. Sistem ohranja uporabniški prostor in aplikacijski model sistema Android, hkrati pa odstrani proizvajalčevo preobleko in poveča preglednost sistema. Pričakovati je mogoče, da bo LineageOS v primerjavi z mobilnimi distribucijami sistema GNU/Linux uporabniško bliže tovarniškemu sistemu, vendar bo še vedno ostal odvisen od istih omejitev spodnjih slojev strojne podpore.

### 6.3.3 Ubuntu Touch

Ubuntu Touch je v tej študiji predstavnik uporabniško usmerjenega mobilnega sistema GNU/Linux. Uradna stran naprav projekta Ubuntu Touch za OnePlus 6T navaja, da je naprava na voljo v installerju, da večina ključnih funkcij deluje ter da je sistem večinoma primeren za vsakodnevno rabo z nekaterimi manjšimi omejitvami [73].

Za diplomsko nalogo je Ubuntu Touch pomemben zato, ker poskuša ponuditi zaokrožen mobilni operacijski sistem zunaj ekosistema Google Android. V primerjavi z LineageOS nas zato zanima predvsem, kako visoka je cena prehoda iz uporabniškega prostora sistema Android v mobilni sistem GNU/Linux. Posebej pomembna bodo opažanja pri aplikacijskem ekosistemu, telefoniji, upravljanju porabe energije in splošni odzivnosti uporabniškega vmesnika.

### 6.3.4 postmarketOS

postmarketOS je v tej študiji predstavnik bolj odprtega in eksperimentalnega pola mobilnega sistema GNU/Linux. Projekt vodi OnePlus 6T kot skupnostno napravo `oneplus-fajita`, posebna stran naprave pa opisuje stanje podpore in status glavne razvojne veje jedra Linux za to napravo [54, 52].

V primerjavi z Ubuntu Touch je postmarketOS bliže logiki klasične distribucije GNU/Linux. To pomeni večji poudarek na paketnem upravljanju, terminalu, standardnem uporabniškem prostoru in dolgoročnem vzdrževanju.

Hkrati pa je pri takem sistemu smiselno pričakovati več praktičnih omejitev pri funkcijah, ki so na komercialnih telefonih močno odvisne od dobaviteljskih komponent in specifičnih prilagoditev za Android.

## 6.4 Metodologija in potek testiranja

Da bi bili rezultati med sistemi čim bolj primerljivi, testiranje izvajamo v čim bolj enotnih pogojih. Pri vseh sistemih, kjer je to mogoče, uporabimo isto napravo, isto kartico SIM, isto brezžično omrežje, primerljivo svetlost zaslona in enak nabor osnovnih uporabniških opravil. Po vsaki namestitvi sistem pustimo v osnovni konfiguraciji, nato pa dodamo le tiste aplikacije in nastavitve, ki so potrebne za primerljiv preizkus vsakdanje rabe.

Popolna simetrija med sistemi ni vedno dosegljiva. Na sistemih GNU/Linux enake aplikacije kot na sistemu Android pogosto niso na voljo, zato tam, kjer je to potrebno, uporabimo funkcionalno najbližjo alternativo ali spletno različico storitve. Cilj ni umetno izenačiti različnih platform, temveč preveriti, ali posamezen sistem omogoča razumno izvedbo iste uporabniške naloge.

Zaradi bolj poštene primerjave se izognemo sintetičnim merilnikom zmogljivosti, ki niso enako dostopni ali enako smiselni na vseh štirih sistemih. Namesto tega se osredotočimo na ponovljive scenarije dejanske rabe. Tak pristop je bolj skladen z osrednjim vprašanjem naloge, saj nas zanima predvsem, ali je sistem uporaben kot telefon za vsakdanjo rabo, ne pa le, kako visoko doseže rezultat v posameznem merilniku.

### 6.4.1 Splošna pravila testiranja

Pri vseh testih uporabljamo vnaprej določen protokol. Za vsak sistem zabeležimo datum začetka testiranja, točno različico sistema, način namestitve, vrsto začetne konfiguracije ter morebitne posebnosti, ki bi lahko vplivale na rezultate.

Osnovne funkcionalne preizkuse izvedemo vsaj dvakrat. Če se rezultat

med poskusoma razlikuje, test ponovimo še tretjič in kot končni rezultat upoštevamo prevladujoče stanje, odstopanje pa posebej zabeležimo. Pri funkcijah, kjer je to smiselno, poleg samega uspeha ali neuspeha zabeležimo tudi kakovost delovanja, na primer stabilnost, odzivnost in uporabniško zaznavne omejitve.

Pri vseh sistemih poskušamo ohraniti čim bolj primerljive pogoje: isto napravo, isto kartico SIM, isto brezžično omrežje, primerljivo svetlost zaslona in enak testni nabor opravil. Kadar popolna primerljivost ni mogoča, to posebej navedemo v rezultatih.

Pri osnovnih opravilih imajo prednost prednaložene oziroma privzeto ponujene aplikacije sistema, saj te najbolje predstavljajo privzeto uporabniško izkušnjo konkretne rešitve. Če privzeta aplikacija za določeno nalogo ni na voljo, ni dovolj stabilna ali naloge ne omogoča na razumno uporaben način, uporabimo najbolj smiselno dodatno alternativo. Tak primer v rezultatih posebej označimo in upoštevamo kot praktično omejitev sistema.

Pri aplikacijskem ekosistemu damo prednost uradnim aplikacijam oziroma uradnim spletnim različicam storitev. Če te niso na voljo, lahko izjemoma uporabimo najbolj uveljavljeno skupnostno rešitev, vendar jo v rezultatih jasno označimo kot nepopoln približek uradni podpori. Takih rešitev ne obravnavamo kot povsem enakovrednih uradnim aplikacijam.

Pri preizkusih avtonomije in zmogljivosti uporabimo enake časovne intervale in enake obremenitve. Pred posameznim testom po možnosti zapremo odprte aplikacije, onemogočimo nebistvene dodatne procese ter zabeležimo začetno stanje baterije.

### 6.4.2 Scenariji testiranja

Scenariji testiranja pokrivajo iste funkcionalne sklope praktične uporabnosti, ki smo jih opredelili v poglavju 4. Pri vsakem sklopu nas ne zanima le, ali funkcija deluje, temveč tudi, ali je njena uporaba dovolj zanesljiva za vsakdanjo rabo.

**Osnovne funkcije telefona.** Pri vsakem sistemu preverimo odhodne in dohodne klice, pošiljanje in prejemanje sporočil SMS, delovanje mobilnih podatkov ter možnost souporabe povezave prek dostopne točke.

**Klici.** Izvedemo vsaj dva odhodna in dva dohodna klica, vsak v trajanju najmanj ene minute. Preverimo vzpostavitev klica, kakovost zvoka, stabilnost povezave ter delovanje zvočnika in slušalke med klicem.

**SMS.** Preverimo pošiljanje in prejemanje običajnega kratkega sporočila ter daljšega sporočila s šumniki.

**Mobilni podatki in dostopna točka.** Preverimo vzpostavitev povezave ter stabilno uporabo za osnovna opravila, kot so odpiranje spletnih strani, prenos sporočil in uporaba navigacije. Pri dostopni točki preverimo uspešen vklop in dejansko uporabnost povezave na drugi napravi.

**Ključna strojna podpora.** V tej skupini preverimo Wi-Fi, Bluetooth, GPS, NFC, zvok, kamero, senzorje ter čitalnik prstnih odtisov, kjer je ta funkcionalnost na voljo.

**Wi-Fi in Bluetooth.** Pri Wi-Fi preverimo uspešno povezavo z znanim brezžičnim omrežjem in stabilnost povezave med običajno uporabo. Pri Bluetooth preverimo povezavo z vsaj eno zunanjo napravo in stabilnost prenosa zvoka.

**GPS.** Preverimo vklop lokacijskih storitev, čas do pridobitve prve lokacije, praktično natančnost ter stabilnost pri navigaciji. Kot uspešen rezultat štejemo takšno delovanje, ki omogoča razumno zanesljivo uporabo navigacije brez izrazitega zamika ali pogostega izgubljanja položaja.

**Kamera in galerija.** Pri kameri uporabimo standardiziran nabor preizkusov: fotografiranje v dobri dnevni svetlobi, fotografiranje v slabši notranji

svetlobi, fotografiranje bližnjega motiva, snemanje vsaj 30-sekundnega videa ter zagon aplikacije iz hladnega stanja. Pri tem spremljamo čas zagona aplikacije, stabilnost delovanja, osnovno kakovost izhoda in nabor razpoložljivih funkcij. Pri galeriji preverimo odpiranje fotografij in videov, preklapljanje med posnetki, brisanje, deljenje in splošno odzivnost.

**Senzorji, NFC in čitalnik prstnih odtisov.** Pri senzorjih preverimo vsaj vrtenje zaslona, senzor bližine in samodejno svetlost, kjer je na voljo. Pri NFC preverimo osnovno branje NFC značk, vendar tega ne enačimo avtomatično s podporo za mobilno plačevanje. Kjer je na voljo, preverimo tudi delovanje čitalnika prstnih odtisov.

**Avtonomija in zmogljivost.** Porabo energije ocenjujemo s preprostimi, ponovljivimi vzorci rabe.

**Pripravljenost čez noč.** Pri testu pripravljenosti čez noč pustimo zaslon izklopljen približno osem ur; vse aplikacije v ozadju so pred začetkom zaprte, vključeni pa ostanejo Wi-Fi, mobilni podatki in GPS. Zabeležimo začetni in končni odstotek baterije ter morebitne posebnosti med izvajanjem.

**Brskanje po spletu in predvajanje videa.** Izvedemo 30-minutno brskanje po spletu in 30-minutno predvajanje lokalno shranjenega videa. Pri testu brskanja po spletu uporabimo enako svetlost zaslona in vnaprej določen nabor spletnih strani. Pri testu predvajanja videa uporabimo enak lokalno shranjen video posnetek na vseh sistemih.

**Splošna odzivnost.** Poleg porabe energije opisno zabeležimo tudi segrevanje naprave, splošno odzivnost sistema in morebitne težave, kot so neobičajno hitro praznjenje, nestabilno stanje pripravljenosti ali počasno prebujanje iz spanja.

**Ekosistem aplikacij.** Ker je uporabnost telefona danes tesno povezana z aplikacijami, preverimo, ali posamezen sistem pokrije minimalni nabor vsakodnevnih opravil. Posamezno kategorijo obravnavamo kot pokrito le, če je mogoče nalogo opraviti na razumno zanesljiv in uporabniško sprejemljiv način.

**Spletni brskalnik.** Uporabimo vnaprej določen nabor spletnih strani: `wikipedia.org`, `rtvslo.si`, `github.com`, `reddit.com` in `youtube.com`. Pri vsaki strani preverimo uspešnost nalaganja, osnovno pravilnost prikaza, odzivnost, uporabo obrazcev oziroma prijave, delo z zavihki ter osnovne interakcije, kot so kopiranje besedila, odpiranje povezav in predvajanje videa, kjer je to smiselno.

**E-pošta.** Preverimo uporabo enega računa, ki podpira protokola IMAP in SMTP. Preverimo prijavo, sinhronizacijo, pošiljanje in prejemanje sporočila ter delovanje obvestil.

**Sporazumevanje.** Uporabimo tri vnaprej določene storitve: Telegram, Signal in Facebook Messenger. Telegram uporabimo kot predstavnika bolj odprte in manj vezane komunikacijske platforme, Signal kot predstavnika varnostno usmerjene storitve, Facebook Messenger pa kot predstavnika razširjene zaprte platforme. Pri vsaki storitvi preverimo prijavo, pošiljanje, prejemanje in obvestila. Če je storitev dostopna le prek skupnostnega odjemalca ali spletne različice, to posebej navedemo.

**Navigacija.** Preverimo iskanje cilja, izračun poti, začetek vodenja, odzivnost aplikacije ter stabilnost prikaza položaja med vožnjo ali hojo. Posebej zabeležimo tudi razumljivost in uporabnost navodil za pot.

**Mediji in 2FA.** Pri predvajanju medijev preverimo lokalno glasbo, lokalni video in spletni video, pri čemer spremljamo stabilnost delovanja, morebitno segrevanje in splošno uporabnost. Pri orodjih za avtentikacijo preverimo

uporabo TOTP. V aplikacijo uvozimo testno QR kodo in preverimo, ali sistem pravilno generira pričakovano kodo z uporabo preverjevalne strani `https://2fas.com/auth/check-token/`.

**Bančne aplikacije in mobilno plačevanje.** Posebej zabeležimo tudi stanje pri aplikacijah z višjimi varnostnimi zahtevami, kot so bančne aplikacije in mobilno plačevanje, saj so te v praksi pogosto med prvimi pokazatelji omejitev alternativnih sistemov.

**Prilagodljivost in uporabniški nadzor.** Kot dodatni praktični sklop preverimo tudi tiste vidike, ki so za ciljni profil uporabnika posebej pomembni: dostop do terminala, razpoložljivost paketnega upravljalnika, možnost nameščanja dodatne odprtokodne programske opreme, odstranjevanje nezaželenih aplikacij, preglednost sistema ter možnosti nadzora nad procesi, storitvami in dovoljenji. Ta sklop ni ločen od praktične uporabnosti, temveč jo v našem kontekstu dopolnjuje. Prav večji nadzor nad sistemom je eden glavnih razlogov, zakaj uporabnik sploh poseže po odprtokodni alternativi.

### 6.4.3 Metrike in beleženje rezultatov

Rezultate beležimo na dveh ravneh. Na prvi ravni za posamezne funkcionalnosti uporabimo neposreden opis stanja: "deluje", "deluje delno" ali "ne deluje". Oceno "deluje" dodelimo takrat, ko funkcionalnost v običajnem scenariju uporabe deluje zanesljivo in brez pomembnih omejitev. Oceno "deluje delno" dodelimo, kadar funkcionalnost deluje, vendar z opaznimi omejitvami, slabšo stabilnostjo, zmanjšano zmogljivostjo ali drugimi praktično pomembnimi pomanjkljivostmi. Oceno "ne deluje" dodelimo, kadar funkcionalnosti ni mogoče smiselno uporabiti za predvideni namen. Tak zapis je posebej primeren za osnovne komponente, kot so kamera, Wi-Fi, Bluetooth, GPS in klici, saj omogoča jasen pregled nad tem, kaj je na napravi dejansko uporabno.

Kadar je to pomembno, osnovno oznako dopolnimo s kratko kvalitativno

opombo. Na ta način ohranimo preglednost rezultatov, ne da bi izgubili pomembne praktične podrobnosti.

Na drugi ravni opažanja pretvorimo v sintezne ocene po istem modelu, kot smo ga uvedli v prejšnjih poglavjih. Odprtost ocenjujemo po slojih sistema, praktično uporabnost pa po funkcionalnih sklopih. V obeh primerih uporabimo tristopenjsko lestvico z vrednostmi 0, 1 in 2. Na ta način so rezultati empiričnega dela neposredno primerljivi s sinteznimi ocenami iz analitičnega dela naloge.

Pri subjektivnih vidikih, kot so odzivnost uporabniškega vmesnika, stabilnost in splošni vtis, uporabimo kratke opisne opombe. Namesto navidezno natančnih številčnih ocen damo prednost kratkim kvalitativnim ugotovitvam, na primer da je sistem pri vsakdanji rabi "stabilen", "občasno neodziven" ali "primeren le za potrpežljivo eksperimentalno rabo". Tak pristop je v kontekstu diplomske naloge primernejši kot pretirano natančno točkovanje subjektivnih občutkov.

Pri porabi energije zabeležimo odstotek baterije na začetku in na koncu posameznega preizkusa, pri preizkusu pripravljenosti čez noč pa tudi padec baterije v mirovanju. Pri sistemih, kjer je to mogoče, zabeležimo tudi sistemske informacije o porabi energije ali ozadnih procesih. Ker ti podatki med sistemi niso nujno neposredno primerljivi, služijo predvsem kot dopolnilna interpretativna informacija.

## 6.5 Empirični indikatorji odprtosti

Čeprav odprtosti ni mogoče v celoti izmeriti z neposrednim uporabniškim testom, tudi v empiričnem delu preverimo nekaj konkretnih indikatorjev, ki posredno odražajo kriterije odprtosti iz modela iz prejšnjega poglavja. Pri tem ne ocenjujemo abstraktnega projekta, temveč konkretno kombinacijo projekta in naprave. Indikatorji zato niso zamišljeni kot ločen nadomestni model odprtosti, temveč kot praktičen način, kako na konkretni napravi opazovati nekatere vidike zagonske verige, sistemskih storitev, distribucije pro-

gramske opreme, odvisnosti od zaprtih komponent in dejanske uporabniške avtonomije.

**Zagonska veriga in namestitev.** Zabeležimo, kako dokumentiran je postopek namestitve, ali je odklep zagonskega nalagalnika preprost in ponovljiv, ali sistem zahteva posebne obhode ter kako predvidljivo poteka posodabljanje po namestitvi.

**Sistemske storitve in uporabniški prostor.** Preverimo, koliko nadzora ima uporabnik nad prednameščenimi komponentami, ali je na voljo terminal, ali obstaja paketni upravljalnik oziroma drug pregleden mehanizem nameščanja programske opreme ter kako pregledna je distribucija aplikacij in posodobitev.

**Odvisnost od lastniških komponent in storitev.** Posebej zabeležimo, ali sistem za osnovno delovanje ostaja tesno vezan na lastniške storitve, posebne dobaviteljske komponente ali druge elemente, nad katerimi uporabnik nima dejanskega nadzora. Pri tem upoštevamo tudi, ali so ključne vsakdanje funkcije odvisne od zaprtih ali težko zamenljivih delov sklada.

**Odprta koda in preglednost izvora.** Preverimo, ali so izvorna koda sistema, privzetih aplikacij in ključnih komponent javno dostopne, ali so repozitoriji jasno navedeni ter ali lahko uporabnik razumno ugotovi, iz katerih virov prihajajo sistemske slike, posodobitve in aplikacije. Pri tem nas ne zanima popolna tehnična verifikacija binarnih gradenj, temveč predvsem praktična preglednost izvora programske opreme in jasnost ločitve med odprtimi in zaprtimi deli sistema.

**Uporabniška avtonomija.** Preverimo, ali lahko uporabnik sistem razmeroma samostojno pregleduje, prilagaja in vzdržuje brez nestandardnih obhodov ter brez pretirane odvisnosti od proizvajalca ali projekta samega.

## 6.6 Omejitve študije

Empirični del ima več pomembnih omejitev. Prvič, testirana je ena sama naprava, zato rezultatov ne moremo neposredno posplošiti na celoten mobilni ekosistem. Drugič, rezultati so vezani na konkretne izdaje sistemov, ki so bile v času testiranja na voljo. Tretjič, del ocene, zlasti pri uporabniški izkušnji, neizogibno vsebuje tudi subjektivno komponento.

Dodatna omejitev izhaja iz dejstva, da vse platforme nimajo enakega aplikacijskega okolja. Tam, kjer je na voljo le spletna alternativa ali funkcionalno podoben program, primerjava ni popolnoma simetrična. Vendar to ni zgolj metodološka slabost, temveč tudi del same ugotovitve: če posamezen sistem določenega vsakdanjega opravila ne omogoča na enako neposreden način, je prav to pomemben rezultat praktične primerjave.

Kljub navedenim omejitvam takšna študija primera omogoča dovolj konkreten vpogled v to, kako se na isti napravi dejansko razlikujejo tovarniški sistem Android, alternativni ROM za sistem Android in mobilne distribucije sistema GNU/Linux. Prav ta neposredna primerjava je ključna za odgovor na raziskovalni vprašanji, ki zadevata praktično uporabnost in dejansko izvedljivost skoraj popolnoma odprtokodnega telefona za tehnično bolj podkovanega uporabnika.

# Literatura


[1] Abbas Acar in sod. "50 Shades of Support: A Device-Centric Analysis of Android Security Updates". V: *Proceedings of the Network and Distributed System Security Symposium (NDSS) 2024*. Velik empirični pregled uradnih zapisov posodobitev (2014–2023); pokaže razlike med OEM-i, zamude in omejeno trajanje varnostne podpore pri delu naprav. 2024. DOI: `10.14722/ndss.2024.24175`. URL: `https://www.ndss-symposium.org/wp-content/uploads/2024-175-paper.pdf` (pridobljeno 14. 1. 2026).

[2] Android. *Android Enterprise Recommended Requirements*. Zahteve programa vključujejo tudi "Support current shipping release + one major OS upgrade" in zahteve glede varnostnih posodobitev. n.d. URL: `https://www.android.com/enterprise/recommended/requirements/` (pridobljeno 14. 1. 2026).

[3] Android Developers. *Overview of the Play Integrity API*. Okt. 2025. URL: `https://developer.android.com/google/play/integrity/overview` (pridobljeno 13. 1. 2026).

[4] Android Developers. *SafetyNet (deprecated): The SafetyNet Attestation API is deprecated and has been replaced by the Play Integrity API*. Opozorilo na strani navaja zamenjavo SafetyNet Attestation API s Play Integrity API. Jan. 2026. URL: `https://developer.android.com/privacy-and-security/safetynet` (pridobljeno 13. 1. 2026).

[5] Android Open Source Project. *Android shared system image*. Opis Project Treble: ločitev strojno-specifičnega *vendor* dela in generičnega OS

dela ter vmesnik VINTF. Dec. 2025. URL: https://source.android.com/docs/core/architecture/partitions/shared-system-image (pridobljeno 14. 1. 2026).

[6] Android Open Source Project. *AOSP overview*. Opis Androida kot odprtokodnega programskega sklada za različne tipe naprav. 2025. URL: https://source.android.com/docs/setup/about (pridobljeno 19. 11. 2025).

[7] Android Open Source Project. *Interface versioning*. Opisuje mejo system.img/vendor.img pri Treble in zahtevo po verzioniranih vmesnikih. Avg. 2024. URL: https://source.android.com/docs/core/architecture/hidl/versioning (pridobljeno 14. 1. 2026).

[8] Android Open Source Project. *Mainline (modular system components)*. Opis modularnih posodobitev (Mainline) prek Google Play system update ali OTA mehanizmov partnerjev. Dec. 2025. URL: https://source.android.com/docs/core/ota/modular-system (pridobljeno 14. 1. 2026).

[9] Android Open Source Project. *Verified Boot*. Dokumentacija mehanizma Android Verified Boot (AVB). 2024. URL: https://source.android.com/docs/security/features/verifiedboot (pridobljeno 16. 2. 2026).

[10] *Apache License, Version 2.0*. Apache Software Foundation. URL: https://www.apache.org/licenses/LICENSE-2.0 (pridobljeno 4. 3. 2026).

[11] Apple. *App code signing process in iOS, iPadOS, tvOS, watchOS, and visionOS*. Apple Platform Security / Security Guide. Dec. 2024. URL: https://support.apple.com/guide/security/app-code-signing-process-sec7c917bf14/web (pridobljeno 14. 1. 2026).

[12] Apple Developer. *App Review Guidelines*. Nov. 2025. URL: https://developer.apple.com/app-store/review/guidelines/ (pridobljeno 14. 1. 2026).

[13] François Beaufort. *Media updates in Chrome 62*. Chrome for Developers; razlaga Widevine L1/L3 in povezavo z TEE. Sep. 2017. URL: https://developer.chrome.com/blog/media-updates-in-chrome-62 (pridobljeno 14. 1. 2026).

[14] Bitmovin. *Widevine Security Levels in Depth*. Sekundarni industrijski vir; opisuje tipično povezavo L1 → HD/UHD, L3 → SD. n.d. URL: https://developer.bitmovin.com/playback/docs/widevine-security-levels-in-web-video-playback (pridobljeno 14. 1. 2026).

[15] Conner Bradley. "Sustainable Security: Exploring Longevity Challenges and Solutions for IoT". V razdelku o dolgoživosti avtor izrecno omenja, da projekti, kot je LineageOS, podaljšujejo programsko podporo za naprave brez uradne podpore, npr. z portanjem novejših izdaj Androida na starejše telefone. Master's thesis. Ottawa, Ontario: Carleton University, 2023. URL: https://www.cisl.carleton.ca/~cbradley/data/papers/MastersThesis.pdf (pridobljeno 15. 1. 2026).

[16] Andrew Davis. "Cellular Baseband Security". Bachelor's thesis. Georgia Institute of Technology, 2012. URL: https://repository.gatech.edu/bitstreams/8cb73e2d-184d-41a3-8069-1d21d48ccc9a/download (pridobljeno 14. 1. 2026).

[17] Debian Wiki contributors. *Mobian/Devices*. Debianova dokumentacija za Mobian; navaja uradno podprte androidne naprave, med drugim OnePlus 6/6T in Pocophone F1. Sep. 2025. URL: https://wiki.debian.org/Mobian/Devices (pridobljeno 16. 3. 2026).

[18] e Foundation. *Different Build Types — /e/OS documentation*. Razlaga tipov gradenj Official, Community, Test, Unofficial in Custom. n.d. URL: https://doc.e.foundation/build-status (pridobljeno 16. 3. 2026).

[19] e Foundation. *Smartphone Selector — /e/OS documentation*. Seznam podprtih naprav in oznake Official, Community ter OTA. n.d. URL: https://doc.e.foundation/devices (pridobljeno 16. 3. 2026).

[20] Free Software Foundation. *What is Free Software?* Opredelitev proste programske opreme in štiri temeljne svoboščine. Jan. 2026. URL: https://www.gnu.org/philosophy/free-sw.html (pridobljeno 16. 2. 2026).

[21] FuriLabs. *FLX1s.* Uradna predstavitev naprave FLX1s in sistema FuriOS na osnovi Debiana. Jan. 2026. URL: https://furilabs.com/shop/flx1s/ (pridobljeno 16. 3. 2026).

[22] FuriLabs. *FuriPhone FLX1s Linux Phone.* Uradna stran projekta; poudari tri strojna stikala za zasebnost. n.d. URL: https://furilabs.com/ (pridobljeno 16. 3. 2026).

[23] *GNU General Public License, version 2.* Free Software Foundation. URL: https://www.gnu.org/licenses/old-licenses/gpl-2.0.en.html (pridobljeno 4. 3. 2026).

[24] Google for Developers. *Widevine DRM: Overview.* Okt. 2024. URL: https://developers.google.com/widevine/drm/overview (pridobljeno 14. 1. 2026).

[25] Google Play Console Help. *Use the Play Integrity API to detect risky interactions and fight abuse.* Uradna dokumentacija za razvijalce v Play Console; opisuje signale/verdict (npr. "genuine and certified Android device"). n.d. URL: https://support.google.com/googleplay/android-developer/answer/11395166?hl=en (pridobljeno 15. 1. 2026).

[26] GrapheneOS Project. *CLI install guide — Install — GrapheneOS.* Uradni vodič za namestitev na uradno podprte naprave. n.d. URL: https://grapheneos.org/install/cli (pridobljeno 16. 3. 2026).

[27] GrapheneOS Project. *Frequently Asked Questions — GrapheneOS.* Uradni FAQ; podprte naprave, razlogi za ozek nabor podpore in dolžina podpore. n.d. URL: https://grapheneos.org/faq (pridobljeno 16. 3. 2026).

[28] GrapheneOS Project. *GrapheneOS: the private and secure mobile OS.* Glej razdelek "About". 2026. URL: https://grapheneos.org/ (pridobljeno 13. 1. 2026).

[29] Halium Project. *Halium – Halium documentation.* Uradni porting guide; opiše Halium kot projekt za poenotenje HAL za GNU/Linux sisteme na napravah s prednameščenim Androidom. 2023. URL: https://docs.halium.org/ (pridobljeno 14. 1. 2026).

[30] Grant Hernandez in sod. "FIRMWIRE: Transparent Dynamic Analysis for Cellular Baseband Firmware". V: *Network and Distributed System Security (NDSS) Symposium 2022.* 2022. DOI: 10.14722/ndss.2022.23136. URL: https://www.cise.ufl.edu/~butler/pubs/ndss22-firmwire.pdf (pridobljeno 14. 1. 2026).

[31] Jolla. *Jolla C2 Community Phone.* Uradna predstavitev referenčne naprave Jolla C2 Community Phone. n.d. URL: https://commerce.jolla.com/products/jolla-community-phone (pridobljeno 16. 3. 2026).

[32] Jolla. *Jolla C2 Community Phone.* Stran navaja podporo za Android aplikacije in možnosti, kot je microG. n.d. URL: https://commerce.jolla.com/products/jolla-community-phone (pridobljeno 16. 3. 2026).

[33] Liberux. *Liberux – Privacy · Security · Freedom.* Uradna predstavitev projekta Liberux NEXX in sistema LiberuxOS. 2026. URL: https://liberux.net/ (pridobljeno 16. 3. 2026).

[34] Liberux. *Liberux NEXX is now live on crowdfunding!* Objava o crowdfundingu projekta Liberux NEXX. 2025. URL: https://liberux.net/category/mastodon/ (pridobljeno 16. 3. 2026).

[35] Liberux. *Our first dev-board has arrived!* Objava o prvi razvojni plošči in zgodnji fazi prototipiranja projekta Liberux NEXX. 2025. URL: https://liberux.net/our-first-dev-board-has-arrived/ (pridobljeno 16. 3. 2026).

[36] libhybris contributors. *libhybris: Load Android drivers from regular GNU/Linux processes.* README: libhybris omogoča nalaganje Android (bionic) knjižnic/gonilnikov znotraj procesov, ki uporabljajo npr. glibc ali musl. 2025. URL: https://github.com/libhybris/libhybris (pridobljeno 14. 1. 2026).

[37] LineageOS Project. *Devices — LineageOS Wiki*. Seznam uradno podprtih naprav LineageOS. 2026. URL: https://wiki.lineageos.org/devices/ (pridobljeno 16. 3. 2026).

[38] LineageOS Project. *Frequently Asked Questions — LineageOS Wiki*. Pojasnila o posodobitvah, Vendor SPL in drugih pogostih vprašanjih. 2026. URL: https://wiki.lineageos.org/faq (pridobljeno 16. 3. 2026).

[39] LineageOS Project. *Google apps — LineageOS Wiki*. Pojasni, da Google apps niso del LineageOS in da gre za lastniške aplikacije. 2026. URL: https://wiki.lineageos.org/gapps/ (pridobljeno 16. 3. 2026).

[40] LineageOS Project. *Install LineageOS on OnePlus 6T – fajita*. Uradna navodila za namestitev; marca 2026 so se nanašala na LineageOS 22.2. n.d. URL: https://wiki.lineageos.org/devices/fajita/install/variant1/ (pridobljeno 27. 3. 2026).

[41] LineageOS Project. *LineageOS – LineageOS Android Distribution*. Opis na strani navaja: "A free and open-source operating system for various devices, based on the Android mobile platform." URL: https://lineageos.org/ (pridobljeno 13. 1. 2026).

[42] LineageOS Project. *OnePlus 6T (fajita)*. Uradna stran naprave OnePlus 6T v dokumentaciji projekta LineageOS. n.d. URL: https://wiki.lineageos.org/devices/fajita/variant1/ (pridobljeno 27. 3. 2026).

[43] Iliyan Malchev, Amith Dsouza in Veerendra Bhora. *Treble Plus One Equals Four*. Razlaga, kako arhitektura okoli Treble vpliva na stroške nadgradenj in podporne časovnice pri čipovjih (SoC) ter OEM-ih. Dec. 2020. URL: https://android-developers.googleblog.com/2020/12/treble-plus-one-equals-four.html (pridobljeno 14. 1. 2026).

[44] Mobian Project. *Mobian*. Uradna predstavitev projekta Mobian kot izpeljanke Debiana za mobilne naprave. n.d. URL: https://mobian.org/ (pridobljeno 16. 3. 2026).

[45] Mobian Project. *What is Mobian?* Uradni blog projekta; pojasni cilje projekta in poudari zmanjševanje projektno specifičnih sprememb z njihovim prenosom v izvorne projekte. Feb. 2021. URL: https://blog.mobian.org/posts/2021/02/18/what_is_mobian/ (pridobljeno 16. 3. 2026).

[46] OnePlus. *OnePlus 6T Specifications.* Uradne tehnične specifikacije telefona OnePlus 6T. n.d. URL: https://www.oneplus.com/support/spec/oneplus-6t (pridobljeno 27. 3. 2026).

[47] OnePlus. *OxygenOS 11.1.2.2 for the OnePlus 6 and OnePlus 6T.* Uradna objava zadnje javno objavljene izdaje OxygenOS za OnePlus 6/6T na Androidu 11. Jan. 2022. URL: https://community.oneplus.com/thread/1523813 (pridobljeno 27. 3. 2026).

[48] Open Source Initiative. *The Open Source Definition.* Uradna definicija odprtokodne programske opreme (OSI). Feb. 2024. URL: https://opensource.org/osd (pridobljeno 16. 2. 2026).

[49] PINE64. *PinePhone.* Opis naprave PinePhone, poudari podporo za mainline Linux, strojna stikala za zasebnost ter osnovne specifikacije. n.d. URL: https://pine64.org/devices/pinephone/ (pridobljeno 14. 1. 2026).

[50] PINE64 Wiki. *PinePhone.* Dokumentacija in kontekst (izdaje, stanje programske opreme, namestitev, specifikacije, OS-ji). Nov. 2024. URL: https://wiki.pine64.org/wiki/PinePhone (pridobljeno 14. 1. 2026).

[51] postmarketOS Project. *About postmarketOS.* Opis ciljev in arhitekture postmarketOS kot razširitve distribucije Alpine Linux na pametne telefone. 2024. URL: https://wiki.postmarketos.org/wiki/About_postmarketOS (pridobljeno 19. 11. 2025).

[52] postmarketOS Project. *All devices — postmarketOS Wiki.* Seznam naprav; marca 2026 je bil OnePlus 6T voden kot skupnostna naprava. Mar. 2026. URL: https://wiki.postmarketos.org/wiki/All_devices (pridobljeno 27. 3. 2026).

[53] postmarketOS Project. *Devices — postmarketOS Wiki*. Seznam podprtih naprav; stran je marca 2026 navajala 735 podprtih naprav. Mar. 2026. URL: https://wiki.postmarketos.org/wiki/Devices (pridobljeno 16. 3. 2026).

[54] postmarketOS Project. *OnePlus 6T (oneplus-fajita)*. Wiki stran naprave; opisuje stanje podpore in status glavne razvojne veje jedra Linux za OnePlus 6T. Okt. 2025. URL: https://wiki.postmarketos.org/wiki/OnePlus_6T_%28oneplus-fajita%29 (pridobljeno 27. 3. 2026).

[55] Purism. *Librem 5*. Uradna stran naprave Librem 5 (cilji, zasebnost, osnovne lastnosti in usmeritev v odprtokodnost). n.d. URL: https://puri.sm/products/librem-5/ (pridobljeno 14. 1. 2026).

[56] Purism. *Privacy in Depth*. Pojasni Lockdown mode in izklop dodatnih senzorjev na Librem 5. Jul. 2022. URL: https://puri.sm/posts/privacy-in-depth/ (pridobljeno 16. 3. 2026).

[57] Purism. *Security*. Opis varnostnega modela Librem 5, vključno s Lockdown mode. n.d. URL: https://puri.sm/security/ (pridobljeno 16. 3. 2026).

[58] Purism Documentation. *Librem 5 – Documentation*. Tehnična dokumentacija za Librem 5 (programski sklad, uporaba, komponente). n.d. URL: https://docs.puri.sm/Librem_5/ (pridobljeno 14. 1. 2026).

[59] Replicant Project. *Replicant*. Uradna predstavitev projekta Replicant. n.d. URL: https://replicant.us/about.php (pridobljeno 16. 3. 2026).

[60] Replicant Project. *Supported devices*. Seznam podprtih in vzdrževanih naprav projekta Replicant. n.d. URL: https://replicant.us/supported-devices.php (pridobljeno 16. 3. 2026).

[61] Reproducible Builds. *Reproducible Builds*. Projekt in smernice za reproducibilne gradnje programskih paketov. n.d. URL: https://reproducible-builds.org/ (pridobljeno 16. 2. 2026).

[62] Sailfish OS Documentation. *Hardware Adaptation Development Kit – libhybris*. Opiše uporabo libhybris kot most med GNU libc uporabniškim prostorom in Android (bionic) komponentami pri strojnih prilagoditvah. 2024. URL: https://docs.sailfishos.org/Tools/Hardware_Adaptation_Development_Kit/ (pridobljeno 14. 1. 2026).

[63] Sailfish OS Documentation. *Open Source and Free Software in Sailfish OS*. Uradna dokumentacija; pojasni, da Sailfish OS vsebuje tako odprtokodne kot tudi proprietary komponente. n.d. URL: https://docs.sailfishos.org/Develop/Open_Source/ (pridobljeno 16. 3. 2026).

[64] StatCounter GlobalStats. *Mobile Operating System Market Share Worldwide*. Dostopno prek StatCounter GlobalStats, podatki za obdobje 2024–2025. 2025. URL: https://gs.statcounter.com/os-market-share/mobile/worldwide (pridobljeno 13. 1. 2026).

[65] Team Counterpoint. *Global Smartphone Sales Share by Operating System*. Counterpoint Research, Insight. Nov. 2025. URL: https://counterpointresearch.com/en/insights/global-smartphone-os-market-share (pridobljeno 13. 1. 2026).

[66] *The 3-Clause BSD License*. Open Source Initiative. URL: https://opensource.org/license/bsd-3-clause (pridobljeno 4. 3. 2026).

[67] The Linux Kernel Archives. *The Linux Kernel Archives*. Uradna stran izdaj jedra Linux in referenca za "mainline". n.d. URL: https://www.kernel.org/ (pridobljeno 16. 2. 2026).

[68] *The MIT License*. Open Source Initiative. URL: https://opensource.org/license/mit (pridobljeno 4. 3. 2026).

[69] Ken Thompson. "Reflections on Trusting Trust". V: *Communications of the ACM* 27.8 (1984), str. 761–763. DOI: 10.1145/358198.358210.

[70] UBports Community. *Convergence*. Uradna UBports dokumentacija; razlaga koncept konvergence (telefon ↔ namizje) in cilje uporabniške izkušnje. 2026. URL: https://docs.ubports.com/en/latest/

humanguide/other-design-considerations/convergence.html (pridobljeno 15. 1. 2026).

[71] UBports Community. *Introduction – UBports documentation*. Uradna dokumentacija projekta UBports/Ubuntu Touch; opis ciljev projekta in kratka zgodovina. 2026. URL: https://docs.ubports.com/en/latest/about/introduction.html (pridobljeno 14. 1. 2026).

[72] UBports Community. *Introduction to porting — UBports documentation*. Pojasni, da je Ubuntu Touch mogoče prenesti na naprave, prvotno dobavljene s sistemom Android, in opozori na odvisnost od lastniške programske opreme. 2026. URL: https://docs.ubports.com/en/latest/porting/introduction/Intro.html (pridobljeno 16. 3. 2026).

[73] UBports Community. *OnePlus 6T – Ubuntu Touch device page*. Uradna stran naprave v katalogu Ubuntu Touch; navaja delujoče funkcije in splošno stopnjo uporabnosti. n.d. URL: https://devices.ubuntu-touch.io/device/fajita/ (pridobljeno 27. 3. 2026).

[74] Widevine Help. *How to determine if Android device is security Level 1 or Level 3*. Google Support; vsebuje tudi status DEVICE_IS_PROVISIONED_SD_ONLY za Widevine L3. n.d. URL: https://support.google.com/widevine/answer/6072714?hl=en (pridobljeno 14. 1. 2026).